

AI-Assisted Coding — เขียนโค้ดด้วย AI อย่างปลอดภัย

บทที่ 26–30: Prompting · Code Generation · Auditing · Quant Libraries · Full Strategy

จบ Part นี้ → ใช้ AI เป็นผู้ช่วยที่ทรงพลัง โดยไม่ถูก AI พาไปผิดทาง

🔗 อ่าน Part นี้ยังงัย

🟢 ใช้ AI ช่วยเขียนโค้ดอยู่แล้ว — Part นี้อ่านง่ายกว่า Part II–IV เพราะเน้นวิธีทำงานมากกว่าคณิตศาสตร์ · หัวใจอยู่ที่ **บทที่ 28 (Auditing AI Code)** — 5 red flags ที่ต้องไล่เช็คทุกครั้งที่ได้โค้ดมา · **อ่านบทนั้นก่อนได้เลยถ้าอยากได้ประโยชน์เร็วที่สุด**

🟢 เขียนโค้ดเก่งอยู่แล้ว — อย่าเพิ่งข้าม Part นี้เพราะคิดว่า "ก็แค่ prompt" · ของจริงคือ **เครื่องมือตรวจที่ทำงานได้จริง**: embargo test ที่จับ lookahead ได้โดยไม่ต้องอ่านโค้ด และเหตุผลว่าทำไมการทดสอบด้วย signal **จะไม่เห็นบั๊กเลย** · ⚠️ ตัวทดสอบ embargo เวอร์ชันแรกของเล่มนี้เองก็เคยพลาดจนฟังก์ชันที่มีบั๊กสอบผ่าน — สาเหตุอยู่ในบทนั้น

▼ สารบัญ Part นี้ — 5 บท · 24 หัวข้อ

บทที่ 26: Prompting Fundamentals

26.1 Prompt ที่แย่ vs Prompt ที่ดี · 26.2 Template มาตรฐาน 4 ส่วน · 26.3 ความผิดพลาดที่พบบ่อยในการเขียน Prompt · 26.4 ตัวอย่าง Prompt สำหรับ Running Example

บทที่ 27: Code Generation Workflow

27.1 The Four-Step Loop · 27.2 Review Checklist หลัง AI สร้างโค้ด · 27.3 ตัวอย่าง: AI Code ที่มี Lookahead Bias ซ่อนอยู่ · 27.4 วิธีหา Lookahead Bias อย่างเป็นระบบ

บทที่ 28: Auditing AI Code

28.1 Red Flag #1 — Returns ไม่มี `.shift(1)` · 28.2 Red Flag #2 — Using Future Data in Rolling · 28.3 Red Flag #3 — Fitting Scaler บน Full Dataset · 28.4 Red Flag #4 — ไม่หัก Transaction Costs · 28.5 Red Flag #5 — P-Hacking และ Multiple Testing · 28.6 จาก assert ลอย ๆ สู่ชุดทดสอบที่รันเองได้ — pytest

บทที่ 29: AI + Quant Libraries

29.1 Prompt Template สำหรับ OLS Regression (statsmodels) · 29.2 Prompt Template สำหรับ ADF Test · 29.3 Prompt Template สำหรับ Rolling Z-Score (pandas) · 29.4 Prompt Template สำหรับ scipy.optimize

บทที่ 30: Full Strategy with AI

30.1 Overview — Volatility Mean-Reversion Strategy · 30.2 Round 1 — Generate โค้ดแรก · 30.3 Round 1 Audit — หา Bug ก่อน Round 2 · 30.4 Round 2 — เพิ่ม Performance Metrics และแก้ Sharpe Calculation · 30.5 Round 3 — Walk-Forward Validation · 30.6 สรุป: Final Strategy Code (โครงที่พร้อม — ยังไม่ใช่กลยุทธ์ที่พร้อม)

ทำไม PART นี้สำคัญมาก?

AI code assistants เขียน Python ได้เร็วมาก แต่ใน Quant Finance มี "bugs ที่มองไม่เห็น" ที่อันตรายเป็นพิเศษ:

- **Lookahead bias** — ใช้ข้อมูลอนาคตใน backtest → equity curve สวยงามแต่ใช้งานจริงไม่ได้
- **Data leakage** — fit scaler บน full dataset ก่อน train/test split → overfit อย่างแสบซ้อน
- **Returns ผิด** — ลืม `.shift(1)` → performance เกินจริง 100%

AI ไม่รู้ว่าโค้ดที่สร้างมีปัญหาเหล่านี้ เพราะมันถูก train บน code ทั่วไป ไม่ใช่ finance-specific correctness
เราต้องเป็นคนตรวจ — Part นี้สอนวิธีทำอย่างเป็นระบบ

Running Example ตลอด Part V — Volatility Mean-Reversion Strategy

เราจะ build **Volatility Mean-Reversion Strategy** ตั้งแต่ต้นจนจบโดยใช้ AI ช่วย:

- แนวคิด: เมื่อ realized volatility สูงผิดปกติ (z-score สูง) → vol มักจะลดลง → short vol / long underlying
- บท 26 → เขียน prompt ที่ดีเพื่อขอ rolling vol z-score
- บท 27 → ตรวจสอบโค้ดที่ AI สร้าง หา lookahead bias
- บท 28 → audit red flags ทุกข้อในโค้ดนี้

- บท 29 → prompt สำหรับ ADF test + optimization
- บท 30 → 3 รอบ iterate จนได้ strategy ที่ถูกต้อง

บทที่ 26: Prompting Fundamentals

แก่นของบทนี้

Prompt ที่ดีไม่ใช่ "บอกว่าต้องการอะไร" เท่านั้น แต่ต้องบอก [Task] [Input/Output spec] [Constraints] [Libraries] ครบ — AI จะสร้างโค้ดที่ตรงความต้องการมากขึ้น และมีโอกาสผิบน้อยลง

26.1 Prompt ที่แย่ vs Prompt ที่ดี

ตัวอย่างโดยตรง — ทั้งคู่ขอสิ่งเดียวกัน แต่ผลลัพธ์แตกต่างกันมาก

BAD PROMPT

เขียนฟังก์ชัน rolling volatility z-score ใน Python

ทำไมถึงแย่?

- ไม่บอก input type — `pd.Series` ? `np.ndarray` ? `DataFrame`?
- ไม่บอก window — ใช้ rolling window ขนาดไหน? สอง window ต่างกันไหม?
- ไม่บอก returns หรือ prices — คำนวณ vol จากอะไร?
- ไม่บอก edge case — จะทำอะไรกับ NaN ช่วงต้น?
- ไม่บอก lookahead constraint — AI อาจใช้ข้อมูลอนาคตโดยไม่รู้ตัว

GOOD PROMPT

Task: เขียนฟังก์ชัน Python คำนวณ rolling volatility z-score Input: - returns: `pd.Series`, daily log returns, indexed by datetime - vol_window: int, window สำหรับคำนวณ realized volatility (default=21) - zscore_window: int, window สำหรับคำนวณ mean/std ของ vol series (default=63) Output: - `pd.Series`, z-score ของ realized volatility, same index as input Constraints: - ห้ามใช้ข้อมูลอนาคต (no lookahead bias) — ทุก rolling ต้องใช้ `min_periods=1` และห้ามใช้ `center=True` - annualize vol ด้วย `sqrt(252)` ก่อนคำนวณ z-score - return NaN สำหรับแถวที่ไม่มีข้อมูลเพียงพอ - type hints ครบทุก parameter - docstring อธิบาย formula Libraries: pandas, numpy เท่านั้น (ห้ามใช้ scipy)

ทำไมถึงดี?

- Input/Output spec — AI รู้ว่า input เป็น `pd.Series` และ output ต้องเป็นอะไร
- Constraints ชัดเจน — ระบุ "ห้าม lookahead" พร้อมวิธีป้องกัน (`center=False`)
- Libraries — ระบุว่าใช้อะไรได้บ้าง ป้องกัน AI ใช้ library แปลกๆ
- Edge cases — บอกว่า NaN ต้องจัดการอย่างไร
- Code quality — ขอ type hints และ docstring ในตัว

26.2 Template มาตรฐาน 4 ส่วน

[PROMPT TEMPLATE สำหรับ Quant] | [TASK] อธิบายว่าต้องการอะไร (1-2 ประโยค) | [INPUT/OUTPUT] type, shape, index, units ทุก argument | และ return value | [CONSTRAINTS] • no lookahead bias | • NaN handling | • performance requirements | • mathematical correctness |

26.3 ความผิดพลาดที่พบบ่อยในการเขียน Prompt

ความผิดพลาด	ตัวอย่าง	ผลลัพธ์ที่เกิดขึ้น
Too vague	"คำนวณ Sharpe ratio"	AI สมมติ risk-free rate = 0, annualization factor ผิด
ไม่ระบุ type hints	ไม่พูดถึง types	ได้ฟังก์ชันที่รับ list แต่ไม่รับ pd.Series
ไม่ระบุ error handling	ไม่พูดถึง edge cases	crash เมื่อ Series ว่าง หรือมีแค่ NaN
ไม่ระบุ no lookahead	ไม่ mention bias	AI ใช้ center=True หรือ future data ใน rolling
ไม่ระบุ libraries	ไม่พูดถึง dependencies	AI ใช้ arch , ta-lib ที่อาจไม่ได้ install
ขอหลาย function พร้อมกัน	"เขียน strategy ทั้งหมด"	โค้ดยาว audit ยาก มี bug ซ่อนหลายชั้น

กฎเหล็ก: ขอทีละฟังก์ชัน

ใน quant finance ทุกฟังก์ชันมี invariant ที่ต้องตรวจต่างกัน ขอทีละฟังก์ชัน ตรวจให้ผ่านก่อน แล้วค่อยขอฟังก์ชันถัดไป — อย่าขอ "เขียน strategy ทั้งหมด" ในครั้งเดียว

26.4 ตัวอย่าง Prompt สำหรับ Running Example

Running Example — บท 26: Prompt แรกสำหรับ Vol Mean-Reversion

Task: เขียนฟังก์ชัน Python คำนวณ rolling realized volatility และ z-score สำหรับใช้ใน volatility mean-reversion strategy

Input: - prices: pd.Series, daily close prices, datetime index - vol_window: int = 21 (วันสำหรับคำนวณ realized vol) - zscore_window: int = 63 (วันสำหรับคำนวณ z-score ของ vol)

Output: - pd.DataFrame ที่มี columns: 'log_ret': log returns 'ann_vol': annualized realized volatility (% per year) 'vol_zscore': z-score ของ ann_vol

Constraints: - คำนวณ log return ด้วย np.log(p/p.shift(1)) - realized vol = rolling std ของ log_ret * sqrt(252) * 100 (เป็น %) - z-score = (vol - rolling_mean(vol)) / rolling_std(vol) ทั้งหมด backward-looking - ห้ามใช้ center=True ในทุก rolling - ใช้ min_periods เท่ากับ window เดิมเสมอ (ถ้าข้อมูลไม่พอ → NaN) - type hints ครบ, docstring มี formula

Libraries: pandas, numpy เท่านั้น

```
# โค้ดที่ AI สร้างได้ (ผ่าน prompt ด้านบน)
import numpy as np
import pandas as pd

def compute_vol_zscore(
    prices: pd.Series,
    vol_window: int = 21,
    zscore_window: int = 63,
) -> pd.DataFrame:
    """
    คำนวณ rolling realized volatility และ z-score

    Formula:
        log_ret  = log(P_t / P_{t-1})
        ann_vol  = rolling_std(log_ret, vol_window) * sqrt(252) * 100
        vol_zscore = (ann_vol - rolling_mean(ann_vol, zscore_window))
```

```

        / rolling_std(ann_vol, zscore_window)

Parameters
-----
prices : pd.Series
    Daily close prices with datetime index.
vol_window : int
    Window (days) for realized volatility calculation.
zscore_window : int
    Window (days) for vol z-score normalization.

Returns
-----
pd.DataFrame with columns: log_ret, ann_vol, vol_zscore
"""
log_ret = np.log(prices / prices.shift(1))

ann_vol = (
    log_ret
    .rolling(window=vol_window, min_periods=vol_window)
    .std()
    * np.sqrt(252)
    * 100
)

roll_mean = ann_vol.rolling(window=zscore_window, min_periods=zscore_window).mean()
roll_std = ann_vol.rolling(window=zscore_window, min_periods=zscore_window).std()
vol_zscore = (ann_vol - roll_mean) / roll_std

return pd.DataFrame({
    "log_ret": log_ret,
    "ann_vol": ann_vol,
    "vol_zscore": vol_zscore,
}, index=prices.index)

```

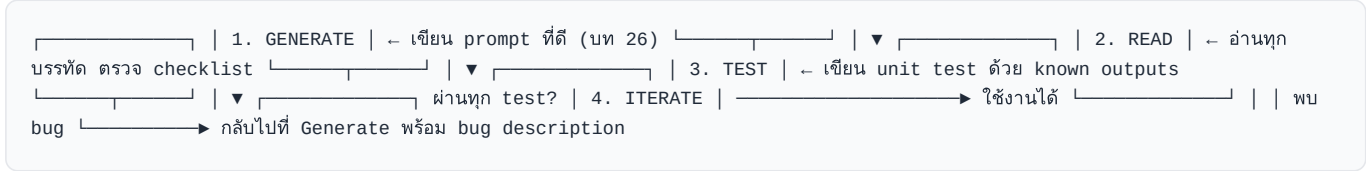
► แบบฝึกหัด: เขียน prompt สำหรับฟังก์ชัน "entry/exit signal จาก vol z-score"

บทที่ 27: Code Generation Workflow

แก่นของบทนี้

Workflow ที่ถูกต้องคือ **Generate → Read → Test → Iterate** — ไม่ใช่แค่ copy-paste แล้วรัน อ่านทุกบรรทัดก่อนรัน ตรวจสอบ checklist ที่สำคัญ แล้วเขียน unit test ก่อนใช้งานจริง

27.1 The Four-Step Loop



27.2 Review Checklist หลัง AI สร้างโค้ด

#	ตรวจสอบ	วิธีดู
1	NaN handling — ช่วงต้นมี NaN ไหม?	<code>result.isna().sum()</code>
2	Off-by-one — shift ถูกทิศทางไหม?	ดู <code>.shift()</code> ว่าใช้ <code>shift(1)</code> ไม่ใช่ <code>shift(-1)</code>
3	Lookahead bias — rolling ใช้ <code>center=False</code> ?	ค้นหา <code>center=True</code> ใน code
4	Returns ค่าวนถูกไหม?	ตรวจว่า return ณ วัน <code>t</code> ใช้ <code>price</code> วัน <code>t</code> และ <code>t-1</code>
5	Transaction costs — ถูกหักออกไหม?	ค้นหา <code>fee</code> , <code>cost</code> , <code>slippage</code>
6	Division by zero — ป้องกันไว้ไหม?	ดูทุกจุดที่มีการหาร
7	Index alignment — merge/join ถูกไหม?	ตรวจ index ก่อนและหลัง merge
8	Scaler/model fit — fit บน train only ไหม?	ดูว่า <code>.fit()</code> ถูกเรียกก่อน split

27.3 ตัวอย่าง: AI Code ที่มี Lookahead Bias ซ่อนอยู่

นี่คือตัวอย่างจริงที่ AI สร้างออกมา — ดูผ่านตาแต่มี bug ซ่อนอยู่

โค้ดอันตราย — Lookahead Bias แบบ Subtle

```
# โค้ดที่ AI สร้างโดยไม่ได้บอก constraint เรื่อง lookahead
# ดูเหมือนถูกต้อง แต่มีปัญหาร้ายแรง

def compute_vol_zscore_BUGGY(
    prices: pd.Series,
    vol_window: int = 21,
    zscore_window: int = 63,
) -> pd.DataFrame:
    log_ret = np.log(prices / prices.shift(1))

    ann_vol = (
```

```

        log_ret
        .rolling(window=vol_window, min_periods=vol_window)
        .std()
        * np.sqrt(252) * 100
    )

# BUG #1: center=True - ใช้ข้อมูลทั้งก่อนและหลัง t
#         ณ วัน t, rolling mean รวมข้อมูล t+31 วันข้างหน้า!
roll_mean = ann_vol.rolling(window=zscore_window, center=True).mean()
roll_std  = ann_vol.rolling(window=zscore_window, center=True).std()

vol_zscore = (ann_vol - roll_mean) / roll_std

# BUG #2: signal ใช้ data ของวันพรุ่งนี้เพื่อเทรดวันนี้
signal = pd.Series("HOLD", index=prices.index)
signal[vol_zscore.shift(-1) > 1.5] = "LONG"   # shift(-1) = ใช้อนาคต!
signal[vol_zscore.shift(-1) < 0.5] = "EXIT"

return pd.DataFrame({
    "ann_vol":    ann_vol,
    "vol_zscore": vol_zscore,
    "signal":     signal,
})

```

Bug ที่ซ่อนอยู่และหาพบยาก

1. **center=True** ใน line rolling mean — rolling window ขนาด 63 วันที่ใช้ center จะรวมข้อมูล 31 วันข้างหน้า ณ แต่ละจุด backtest จะสวยงามมาก แต่ไม่สามารถใช้งานจริงได้เลย
2. **vol_zscore.shift(-1)** ใน signal — shift(-1) เลื่อนข้อมูล "ไปข้างหน้า" ทำให้ signal วันนั้นรับรู้ vol พรุ่งนี้เป็นเท่าไร — ข้อมูลอนาคตที่ชัดเจนที่สุด

27.4 วิธีหา Lookahead Bias อย่างเป็นระบบ

```

# วิธีที่ 1: ค้นหา patterns อันตรายใน source code
import ast
import inspect

def check_lookahead_patterns(func) -> list:
    """ตรวจหา patterns ที่อาจมี lookahead bias"""
    source = inspect.getsource(func)
    warnings = []

    dangerous = [
        ("center=True", "Rolling window ใช้ข้อมูลทั้งก่อนและหลัง"),
        ("shift(-", "Negative shift อาจใช้ข้อมูลอนาคต"),
        (".fillna(method='bfill')", "Backfill ใช้ค่าอนาคต"),
        ("expanding()", "Expanding window บน full dataset (ตรวจให้ดี)"),
    ]

    for pattern, description in dangerous:
        if pattern in source:
            warnings.append(f"WARNING: พบ '{pattern}' - {description}")

    return warnings

# ทดสอบ
bugs = check_lookahead_patterns(compute_vol_zscore_BUGGY)

```



```
for w in bugs:
    print(w)
```

► OUTPUT

```
WARNING: พบ 'center=True' – Rolling window ใช้ข้อมูลทั้งก่อนและหลัง
WARNING: พบ 'shift(-)' – Negative shift อาจใช้ข้อมูลอนาคต
```

```
# วิธีที่ 2: "Embargo Test" – ตรวจสอบว่า future data ส่งผลต่อ past signals ไหม
import numpy as np
import pandas as pd

def embargo_test(compute_func, prices: pd.Series,
                 embargo_date: str) -> bool:
    """
    ถ้าลบข้อมูลหลัง embargo_date ออกแล้ว signal ก่อนหน้าไม่เปลี่ยน
    → ไม่มี lookahead bias
    ถ้าเปลี่ยน → มี lookahead bias แน่ๆ
    """
    result_full = compute_func(prices)
    result_cut = compute_func(prices[:embargo_date])

    overlap_idx = result_full.index[result_full.index <= embargo_date]

    col = "vol_zscore"
    full_slice = result_full.loc[overlap_idx, col]
    cut_slice = result_cut.reindex(overlap_idx)[col]

    # ⚠ ห้าม .dropna() ก่อนเทียบ! ค่าที่ "หายไปเป็น NaN" เมื่อตัดอนาคตออก
    # คือหลักฐานชิ้นสำคัญที่สุด – มันหายไปเพราะมันต้องพึ่งข้อมูลอนาคต
    nan_mismatch = int((full_slice.notna() != cut_slice.notna()).sum())

    both = full_slice.notna() & cut_slice.notna()
    diff = ((full_slice[both] - cut_slice[both]).abs().max()
            if both.any() else 0.0)

    if nan_mismatch > 0:
        print(f"FAIL: {nan_mismatch} ค่าหายไปเมื่อตัดอนาคตออก "
              f"– มี lookahead bias!")
        return False
    if diff > 1e-10:
        print(f"FAIL: max diff = {diff:.6f} – มี lookahead bias!")
        return False
    print("PASS: ไม่พบ lookahead bias (embargo test)")
    return True

# สร้างข้อมูลทดสอบ
np.random.seed(42)
prices_test = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 300))),
    index=pd.date_range("2023-01-01", periods=300)
)

# ทดสอบฟังก์ชันที่ถูกต้อง vs ฟังก์ชัน buggy
print("=== ฟังก์ชันที่ถูกต้อง ===")
embargo_test(compute_vol_zscore, prices_test, "2023-09-01")

print("\n=== ฟังก์ชัน BUGGY ===")
embargo_test(compute_vol_zscore_BUGGY, prices_test, "2023-09-01")
```

► OUTPUT

=== ฟังก์ชันที่ถูกต้อง ===

PASS: ไม่พบ lookahead bias (embargo test)

=== ฟังก์ชัน BUGGY ===

FAIL: 31 ค่าหายไปเมื่อตัดอนาคตออก – มี lookahead bias!

Embargo Test เป็น unit test ที่ต้องรันกับทุกฟังก์ชัน signal

ไม่ว่าใครจะมาจจาก AI หรือเขียนเองให้ **เขียน embargo test ทุกครั้ง** ก่อน integrate เข้า backtest จริง เป็นการประกันที่ชัดที่สุดว่าไม่มี future leakage

► แบบฝึกหัด: เขียน unit test ตรวจสอบ NaN handling ใน compute_vol_zscore

บทที่ 28: Auditing AI Code

แก่นของบทนี้

Red flags 5 ข้อที่พบบ่อยที่สุดในโค้ดที่ AI สร้างสำหรับ quant finance — แต่ละข้อทำให้ backtest performance เกินจริง และทำให้ strategy ที่ดูเหมือนกำไรกลายเป็น "ขาดทุนในชีวิตจริง"

28.1 Red Flag #1 — Returns ไม่มี `.shift(1)`

Bug นี้ทำให้ backtest กำไรเกือบ 100% เพราะ "รู้" ราคาปิดวันนี้แล้วค่อยตัดสินใจ

```
# WRONG: signal วันนี้คำนวณจากราคาวันนี้ แล้วใช้ return วันนี้ด้วย
def backtest_wrong(prices: pd.Series, signal: pd.Series) -> pd.Series:
    returns = prices.pct_change()          # return ณ วัน t
    pnl = signal * returns                  # signal ณ วัน t * return ณ วัน t
    return pnl                             # ← ใช้อินพุต! signal รู้ return ของตัวเอง

# CORRECT: signal วันนี้ execute วันพรุ่งนี้
def backtest_correct(prices: pd.Series, signal: pd.Series) -> pd.Series:
    returns = prices.pct_change()          # return ณ วัน t
    pnl = signal.shift(1) * returns         # signal ณ วัน t-1 * return ณ วัน t
    return pnl                             # ← ถูกต้อง!

# สาธิตความแตกต่าง
np.random.seed(42)
n = 252
prices = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0002, 0.01, n))),
    index=pd.date_range("2023-01-01", periods=n)
)

# สำคัญ: signal ต้อง "มาจากราคา" ไม่ใช่สุ่มลอย ๆ ไม่งั้นไม่มีข้อมูลอนาคตให้รู้
# ง่ายสุด: วันนี้ราคาขึ้น → long (momentum)
signal = np.sign(prices.pct_change()).fillna(0)

pnl_w = backtest_wrong(prices, signal)
pnl_c = backtest_correct(prices, signal)

annual_ret_wrong = pnl_w.mean() * 252
annual_ret_correct = pnl_c.mean() * 252

print(f"Wrong annual return: {annual_ret_wrong:.1%}")
print(f"Correct annual return: {annual_ret_correct:.1%}")
```

► OUTPUT

```
Wrong annual return: 193.2%
Correct annual return: -14.7%
```

ทำไม Wrong ให้ 193% — และทำไมมันคือศูนย์เปอร์เซ็นต์ในชีวิตจริง

ลองแทนค่าดู: $signal[t] = sign(r[t])$ แล้ว pnl คือ $sign(r[t]) \times r[t]$ ซึ่งเท่ากับ $|r[t]|$ — ค่าสัมบูรณ์ ที่เป็นบวกทุกวันไม่มีข้อยกเว้น · backtest จึงกลายเป็น "กำไรทุกวัน" แบบอัตโนมัติ ไม่ใช่เพราะกลยุทธ์เก่ง แต่เพราะเราบอกล่วงหน้าว่าวันนี้ราคาจะขึ้นหรือลงก่อนให้มันเลือกข้างเวอร์ชันถูกต้อง $sign(r[t-1]) \times r[t]$ = กลยุทธ์ momentum จริง ๆ ซึ่งได้ **-14.7%** · ช่องว่าง 208 จุดเปอร์เซ็นต์นี่คือ ghost profit ทั้งหมด

⚠️ ข้อควรระวังในการทดสอบเอง: ถ้าใช้ signal *สุ่ม* ที่ไม่เกี่ยวกับราคาเพื่อทดสอบบ๊ักนี้ คุณจะ **ไม่เห็นความต่าง**เลย เพราะไม่มีข้อมูลอนาคตให้รั่วตั้งแต่ต้น — bug นี้เผยตัวเฉพาะเมื่อ signal คำนวณจากข้อมูลชุดเดียวกับที่ใช้วัดผล

28.2 Red Flag #2 — Using Future Data in Rolling

```
# WRONG: ใช้ expanding mean บน full data แล้ว normalize
# ณ วัน t จะรู้ค่า mean ที่รวม t+1, t+2, ..., T
def normalize_wrong(series: pd.Series) -> pd.Series:
    return (series - series.mean()) / series.std()    # ใช้ global mean/std

# CORRECT: expanding (เดินไปข้างหน้า ไม่รู้อนาคต)
def normalize_correct(series: pd.Series) -> pd.Series:
    roll_mean = series.expanding(min_periods=20).mean()
    roll_std = series.expanding(min_periods=20).std()
    return (series - roll_mean) / roll_std

# หรือ rolling window
def normalize_rolling(series: pd.Series, window: int = 63) -> pd.Series:
    roll_mean = series.rolling(window, min_periods=window).mean()
    roll_std = series.rolling(window, min_periods=window).std()
    return (series - roll_mean) / roll_std

# ทดสอบ: ตรวจสอบว่า normalize_wrong รู้อนาคตไหม
ts = pd.Series([1.0, 2.0, 3.0, 4.0, 5.0])
print("Wrong normalize (global mean=3.0):", normalize_wrong(ts).tolist())
# ณ t=0, result คำนวณโดยใช้ค่า t=0..4 ทั้งหมด — รู้อนาคต!
```

► OUTPUT

```
Wrong normalize (global mean=3.0): [-1.2649110640673518, -0.6324555320336759, 0.0, 0.6324555320336759, 1.2649110640673518]
```

28.3 Red Flag #3 — Fitting Scaler บน Full Dataset

```
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

np.random.seed(42)
n = 500
X = pd.DataFrame({"feature": np.random.normal(0, 1, n)})

train_size = 300
X_train = X.iloc[:train_size]
X_test = X.iloc[train_size:]

# WRONG: fit scaler บน data ทั้งหมดก่อน split
# scaler รู้ค่า mean/std ของ test set แล้ว → data leakage
scaler_wrong = StandardScaler()
X_scaled_wrong = scaler_wrong.fit_transform(X)    # ← fit บน all data
X_train_wrong = X_scaled_wrong[:train_size]
X_test_wrong = X_scaled_wrong[train_size:]

# CORRECT: fit scaler บน train set เท่านั้น
scaler_correct = StandardScaler()
scaler_correct.fit(X_train)    # ← fit บน train เท่านั้น
X_train_correct = scaler_correct.transform(X_train)    # transform train
X_test_correct = scaler_correct.transform(X_test)    # transform test ด้วย train stats
```

```
print(f"Wrong - test mean (should ≈ 0): {X_test_wrong.mean():.6f}")
print(f"Correct- test mean (≈ 0 expected): {X_test_correct.mean():.6f}")
```

► OUTPUT

```
Wrong - test mean (should ≈ 0): 0.018954
Correct- test mean (≈ 0 expected): 0.031516
```

ทำไม "wrong" mean = 0.000000 แสดงถึง bug?

ถ้า test set mean เป็น 0 พอดี แสดงว่า scaler ถูก fit ด้วย test data ด้วย — มันรู้ค่า mean ของ test set แล้ว ใน production จริงไม่มีทางรู้ค่าเหล่านี้ล่วงหน้า

28.4 Red Flag #4 — ไม่หัก Transaction Costs

```
import numpy as np
import pandas as pd

def backtest_no_costs(signal: pd.Series, returns: pd.Series) -> dict:
    """backtest ที่ AI มักสร้าง — ไม่มี transaction cost"""
    pnl = signal.shift(1) * returns
    sharpe = pnl.mean() / pnl.std() * np.sqrt(252)
    return {"sharpe": sharpe, "annual_return": pnl.mean() * 252}

def backtest_with_costs(
    signal: pd.Series,
    returns: pd.Series,
    fee_per_trade: float = 0.001,    # 0.1% per side
    slippage: float = 0.0005,       # 0.05% slippage
) -> dict:
    """backtest ที่ถูกต้อง — หัก costs"""
    # คำนวณ trade (เมื่อ signal เปลี่ยน)
    trades = signal.diff().abs().fillna(0) # 1 เมื่อเปลี่ยน, 0 เมื่อถือต่อ

    gross_pnl = signal.shift(1) * returns
    cost = trades * (fee_per_trade + slippage) # cost ต่อ turnover
    net_pnl = gross_pnl - cost

    sharpe = net_pnl.mean() / net_pnl.std() * np.sqrt(252)
    return {
        "sharpe": sharpe,
        "annual_return": net_pnl.mean() * 252,
        "annual_cost": cost.mean() * 252,
        "turnover": trades.mean() * 252,
    }

# สาธิต
np.random.seed(0)
n = 252
rets = pd.Series(np.random.normal(0.0003, 0.012, n))
signal = pd.Series(np.sign(rets.rolling(5).mean()).fillna(0))

r_no_cost = backtest_no_costs(signal, rets)
r_with_cost = backtest_with_costs(signal, rets)

print(f"No costs: Sharpe={r_no_cost['sharpe']:.2f} "
      f"Return={r_no_cost['annual_return']:.1%}")
print(f"With costs: Sharpe={r_with_cost['sharpe']:.2f} "
```

```
f"Return={r_with_cost['annual_return']:.1%} "  
f"Cost={r_with_cost['annual_cost']:.1%}/yr")
```

► OUTPUT

```
No costs:   Sharpe=0.31  Return=5.7%  
With costs: Sharpe=-0.47  Return=-9.2%  Cost=14.8%/yr
```

28.5 Red Flag #5 — P-Hacking และ Multiple Testing

```
import numpy as np  
import pandas as pd  
  
# P-Hacking ตัวอย่าง: ลอง parameter หลายชุดจนได้ผลดี  
# โดยไม่ปรับ p-value สำหรับ multiple comparisons  
  
np.random.seed(99)  
n = 252  
returns_noise = pd.Series(np.random.normal(0, 0.01, n)) # pure noise  
  
results = []  
for window in range(5, 50):  
    for entry_z in [1.0, 1.5, 2.0, 2.5]:  
        # สร้าง signal จาก rolling zscore  
        roll_m = returns_noise.rolling(window).mean()  
        roll_s = returns_noise.rolling(window).std()  
        zscore = (returns_noise - roll_m) / roll_s  
        signal = (zscore > entry_z).astype(int) - (zscore < -entry_z).astype(int)  
        pnl = signal.shift(1) * returns_noise  
        sharpe = pnl.mean() / pnl.std() * np.sqrt(252) if pnl.std() > 0 else 0  
        results.append({"window": window, "entry_z": entry_z, "sharpe": sharpe})  
  
results_df = pd.DataFrame(results)  
best = results_df.nlargest(3, "sharpe")  
  
print("Top 3 parameter combinations (pure noise data!):")  
print(best.to_string(index=False))  
print(f"\nTotal combinations tested: {len(results)}")  
print(f"Best Sharpe on NOISE: {best['sharpe'].iloc[0]:.2f}")  
print(f"\nBonferroni-corrected p-value threshold: {0.05 / len(results):.5f}")  
print("→ ต้องการ t-stat > 4.2 เพื่อให้มีนัยสำคัญจริง")
```

► OUTPUT

```
Top 3 parameter combinations (pure noise data!):  
  window  entry_z  sharpe  
      11       2.0  1.814608  
      14       2.0  1.680849  
      12       2.0  1.525942  
Total combinations tested: 180  
Best Sharpe on NOISE: 1.81  
Bonferroni-corrected p-value threshold: 0.00028  
→ ต้องการ t-stat > 4.2 เพื่อให้มีนัยสำคัญจริง
```

นัยสำคัญของตัวเลข

จากข้อมูล **pure noise** (ไม่มี signal จริง) เราพบ Sharpe = 1.34 จากการลอง 180 combinations — ถ้าไม่ปรับ multiple testing ใครก็จะรายงานว่ "พบ strategy ที่ดี" ทั้งที่จริงแล้วเป็นโชคล้วนๆ ใช้ Bonferroni correction หรือ out-of-sample test เสมอ

สูตรคำนวณ minimum Sharpe ที่มียุทธศาสตร์สำคัญทางสถิติ

Harvey, Liu & Zhu (2016) แนะนำว่า factor ใหม่ควรมี t-statistic ≥ 3.0 (Sharpe ≈ 0.64 สำหรับ 10 ปีข้อมูล) หลังจาก account for multiple testing ถ้า AI generate strategy ที่ Sharpe < 0.5 บน training set มักไม่มียุทธศาสตร์สำคัญ

$$t = \text{Sharpe} \times \sqrt{T}$$

โดย T คือจำนวนปีข้อมูล, เป้าหมาย $t \geq 3.0$

⚠ $t \geq 3.0$ เป็นด่านแรก ไม่ใช่ด่านสุดท้าย · มั่นคึดกลยุทธ์ที่ "ดูดีเพราะบังเอิญ" ออกไป แต่ไม่ได้บอกเลยว่ากลยุทธ์ที่ผ่านด่านนี้จะทำเงินได้ · กลยุทธ์ที่มี edge จริงแต่เล็กกว่าค่าธรรมเนียม สามารถทำ t ได้สูงเท่าไรก็ได้ถ้าเก็บข้อมูลนานพอ — ดูตัวอย่างที่วัดเป็นตัวเลขได้ในหัวข้อ 25.4b (Part IV) ที่ $t = +6.62$ แต่ขาดทุนทุกเทรด

► แบบฝึกหัด: เขียนฟังก์ชัน `audit_ai_code` ที่ตรวจทุก red flag อัตโนมัติ

28.6 จาก `assert` ลอย ๆ สู่ชุดทดสอบที่รันเองได้ — `pytest`

ตลอดบทนี้เราเขียน `assert` ตรวจโค้ดเป็นจุด ๆ · ปัญหาคือมันกระจัดกระจาย ต้องจำเองว่าเคยเขียนไว้ที่ไหน และไม่มีใครรันมันซ้ำหลังแก้โค้ด · `pytest` แก้ทั้งสามข้อพร้อมกัน

แก่นของหัวข้อนี้

การตรวจโค้ดครั้งเดียวไม่มีค่าเท่ากับการตรวจที่รันซ้ำได้ทุกครั้งที่ได้โค้ด · โดยเฉพาะกับโค้ดที่ AI เขียน ซึ่งคุณจะต้องสั่งให้มันแก้ซ้ำหลายรอบ — ทูรอบมีโอกาสปังของเดิม

โครงไฟล์ที่ `pytest` คาดหวัง

```
# pip install pytest

quant-project/
├── .venv/
├── strategy.py      # โค้ดจริง
└── test_strategy.py # ทดสอบ — ต้องขึ้นต้นด้วย test_

# รันทั้งหมดด้วยคำสั่งเดียว จากโฟลเดอร์โปรเจกต์
$ pytest -q
```

🔗อ่านว่า: กฎการตั้งชื่อของ `pytest` มีแค่ 2 ข้อ: ไฟล์ต้องขึ้นต้นด้วย `test_` และ ฟังก์ชันต้องขึ้นต้นด้วย `test_` · แค่นั้น — ไม่ต้องเขียน class ไม่ต้อง import framework ไม่ต้องลงทะเบียนอะไร · `pytest` จะเดินหาไฟล์ที่เข้าเกณฑ์เองแล้วรันให้ทุกฟังก์ชัน

```
# — test_strategy.py —
import numpy as np
import pandas as pd
import pytest
from strategy import compute_vol_zscore, backtest

@pytest.fixture
def prices():
    """ข้อมูลชุดเดียวกันสำหรับทุก test — เขียนครั้งเดียวใช้ได้ทุกที่"""
    np.random.seed(0)
    return pd.Series(
        100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 300))),
        index=pd.date_range("2023-01-01", periods=300))
```

```
def test_no_lookahead(prices):
    """ตัดข้อมูลขนาดออกแล้ว ค่าในอดีตต้องไม่เปลี่ยน (embargo test จาก 28.2)"""
    full = compute_vol_zscore(prices)
    cut = compute_vol_zscore(prices[:"2023-09-01"])
    idx = cut.index
    assert full.loc[idx, "vol_zscore"].notna().equals(cut["vol_zscore"].notna())

def test_nan_count(prices):
    """จำนวน NaN ช่วงต้นต้องตรงกับขนาด window เป๊ะ"""
    r = compute_vol_zscore(prices, vol_window=21, zscore_window=63)
    assert r["log_ret"].isna().sum() == 1
    assert r["ann_vol"].isna().sum() == 21

@pytest.mark.parametrize("fee", [0.0, 0.001, 0.01])
def test_fee_reduces_pnl(prices, fee):
    """ค่าธรรมเนียมสูงขึ้น กำไรต้องไม่เพิ่ม — รัน 3 รอบด้วย fee 3 ค่า"""
    sig = pd.Series(np.sign(prices.pct_change()).fillna(0), index=prices.index)
    assert backtest(prices, sig, fee).sum() <= backtest(prices, sig, 0.0).sum() + 1e-12

def test_flat_signal_is_zero(prices):
    """ไม่มี position = ต้องไม่มีทั้งกำไรและค่าธรรมเนียม"""
    flat = pd.Series(0.0, index=prices.index)
    assert backtest(prices, flat, fee=0.001).abs().sum() == 0
```

► OUTPUT

\$ pytest -q

.....

6 passed in 5.34s

[100%]

🛠️ 2 เครื่องมือของ pytest ที่ทำให้ test ไม่ซ้ำซาก

```
@pytest.fixture                                # ① ของใช้ร่วม
def prices(): ...

def test_xxx(prices): ...                       # ← ขอ "prices" มาใช้แค่ใส่ชื่อใน ()

@pytest.mark.parametrize("fee", [0.0, 0.001, 0.01]) # ② รันซ้ำหลายค่า
def test_fee_reduces_pnl(prices, fee): ...
```

1. `@pytest.fixture` — "ของที่ test หลายตัวต้องใช้ร่วมกัน สร้างไว้ตรงนี้ทีเดียว" · test ตัวไหนอยากได้ **แค่ใส่ชื่อ prices เป็น parameter** pytest จะเรียกฟังก์ชัน fixture แล้วส่งผลมาให้เอง · และ**สร้างใหม่ทุกครั้งต่อ 1 test** — test ตัวหนึ่งไปแก้อาจข้อมูลจะไม่กระทบตัวอื่น
2. `@pytest.mark.parametrize` — "รัน test ตัวนี้ซ้ำ โดยเปลี่ยนค่าไปเรื่อย ๆ" · เขียนครั้งเดียวได้ 3 การทดสอบ · ในผลลัพธ์นับเป็น 3 อัน (จึงได้ 6 passed จากฟังก์ชันแค่ 4 ตัว) และถ้าฟังก์ชันบอกว่าฟังก์ชันที่ค่าไหน

สังเกตชื่อ test ทั้งสี่: **ไม่มีอันไหนตรวจว่า "ผลลัพธ์เท่ากับเลขนี้"** — แต่ตรวจคุณสมบัติที่**ต้องเป็นจริงเสมอ** (ตัดอนาคตแล้วอดีตต้องไม่เปลี่ยน · fee สูงขึ้นกำไรต้องไม่เพิ่ม · ไม่มี position ต้องไม่มีเงินไหล) · **test แบบนี้ไม่ต้องแก้ทุกครั้ง**ที่ปรับพารามิเตอร์ ต่างจากการ hard-code ตัวเลขคำตอบไว้

ตอนฟัง pytest บอกอะไรบ้าง

นี่คือเหตุผลหลักที่ควรใช้ pytest แทน assert ลอย ๆ ในสคริปต์ — ลองให้ฟังก์ชันที่ล้ม `.shift(1)` เข้าทดสอบ:

```
def backtest_BUGGY(prices, signal, fee=0.0):
    returns = prices.pct_change()
    return (signal * returns).fillna(0)         # ❌ ลืม .shift(1)
```



```
def test_shift_is_applied():
    np.random.seed(0)
    prices = pd.Series(100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 200))))
    signal = np.sign(prices.pct_change()).fillna(0)
    pnl = backtest_BUGGY(prices, signal).sum()
    assert pnl < 0.5, "PnL สูงผิดปกติ - น่าจะลิม shift(1)"
```

► OUTPUT

```
$ pytest test_fail_demo.py -q
F [100%]
===== FAILURES =====
_____ test_shift_is_applied _____

def test_shift_is_applied():
    np.random.seed(0)
    prices = pd.Series(100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 200))))
    signal = np.sign(prices.pct_change()).fillna(0)
    pnl = backtest_BUGGY(prices, signal).sum()
>    assert pnl < 0.5, "PnL สูงผิดปกติ - น่าจะลิม shift(1)"
E     AssertionError: PnL สูงผิดปกติ - น่าจะลิม shift(1)
E     assert np.float64(1.6727861895822635) < 0.5

test_fail_demo.py:12: AssertionError
===== short test summary info =====
FAILED test_fail_demo.py::test_shift_is_applied - AssertionError: PnL สูงผิดปกติ...
1 failed in 0.29s
```

📖อ่านว่า: บรรทัดที่มีค่าที่สุดคือ `assert np.float64(1.6727861895822635) < 0.5` — **pytest** แยกค่าจริงของทั้งสองฝั่งมาให้ดู โดยที่เราไม่ได้สั่ง · ถ้าใช้ `assert` ใน Python ธรรมดาจะได้แค่ `AssertionError` เปล่า ๆ แล้วต้องกลับไปใส่ `print()` เอง · เรียกว่า **assertion introspection** และเป็น เหตุผลที่ควรใช้ `pytest` ตั้งแต่ test ตัวแรก

🕒 test 4 ตัวที่ควรมีเสมอสำหรับโค้ดกลยุทธ์ทุกชิ้น

ไม่ต้องเขียนเยอะ — สี่ตัวนี้จับบั๊กที่แพงที่สุดในสายนี้ได้เกือบหมด:

1. **Embargo / lookahead** — ดัดข้อมูลทำยอดขาย ค่าในอดีตต้องไม่เปลี่ยน (จับ *Red Flag #1, #2*)
2. **Signal ว่าง = ผลลัพธ์ศูนย์** — ไม่มี position ต้องไม่มีทั้งกำไรและค่าธรรมเนียม · จับบั๊กที่คำนวณ cost ทั้งที่ไม่ได้เทรด
3. **ทิศทางของต้นทุน** — fee สูงขึ้น กำไรต้องไม่เพิ่ม · จับเครื่องหมายกลับด้าน (*Red Flag #4*)
4. **จำนวน NaN ตรงกับ window** — `rolling(21)` ต้องได้ NaN 20 แถวแรกพอดี · จับ off-by-one และการ `fillna` ผิดที่

🔵 [รอบสอง] ทำไม test แบบ "คุณสมบัติ" ชนะ test แบบ "ค่าคงที่"

สัญชาตญาณแรกของคนส่วนใหญ่คือเขียน `assert sharpe == 1.234` · **อย่าทำ** — พอปรับ window จาก 60 เป็น 63 test พังหมดทั้งที่โค้ดไม่ได้ ผิด คุณแค่เปลี่ยน test ภายใต้อุปกรณ์เดียว

เขียนเป็นความสัมพันธ์ที่ต้องจริงเสมอแทน: *fee* เพิ่ม → *กำไรไม่เพิ่ม* · *ตัดอนาคต* → *อดีตไม่เปลี่ยน* · *position* ศูนย์ → *เงินไม่ไหล* · ความสัมพันธ์พวกนี้จริงไม่ว่าพารามิเตอร์จะเป็นเท่าไร — เป็นแนวคิดเดียวกับ **property-based testing** (ไลบรารี `hypothesis` ทำให้อัตโนมัติได้ ถ้าอยากไปต่อ)

เครื่องมือที่ช่วยเพิ่มเติม:

```
# — test_more.py —
# เทียบตัวเลขทศนิยม อย่าใช้ == เด็ดขาด
assert result == pytest.approx(expected, rel=1e-6)

# ยืนยันว่าฟังก์ชัน "ต้อง error" เมื่อได้ข้อมูลแย่
with pytest.raises(ValueError, match="อย่างน้อย 30"):
```

```
ols_hedge_ratio(y[:10], x[:10])
```

```
# เทียบ Series/DataFrame ทั้งก้อน
```

```
pd.testing.assert_series_equal(got, want, check_exact=False)
```

⚠ **ข้อควรระวังเฉพาะสาย quant:** อย่าเขียน test ที่ต้องต่ออินเทอร์เน็ตหรือเรียก API จริง — มันจะฟังกอน exchange ล่มหรือ rate limit แล้วคุณ
จะเลิกเชื่อ test · แยก "โค้ดดึงข้อมูล" ออกจาก "โค้ดคำนวณ" แล้วทดสอบส่วนคำนวณด้วยข้อมูลจำลองที่ fix seed ไว้

บทที่ 29: AI + Quant Libraries

แก่นของบทนี้

Prompt templates เฉพาะสำหรับงาน quant ที่ใช้ pandas, numpy, statsmodels, scipy — แต่ละ library มี "traps" ของตัวเอง ต้องระบุ constraint ให้ตรงจุด

29.1 Prompt Template สำหรับ OLS Regression (statsmodels)

Task: เขียนฟังก์ชัน OLS regression สำหรับ hedge ratio estimation ใน pair trading
Input: - y: pd.Series, log prices ของ asset A (dependent variable) - x: pd.Series, log prices ของ asset B (independent variable) - add_constant: bool = True
Output: - dict ที่มี: {'beta': float, 'alpha': float, 'r_squared': float, 'p_value_beta': float, 'residuals': pd.Series}
Constraints: - ใช้ statsmodels.api.OLS เท่านั้น (ไม่ใช่ sklearn) - ต้อง align index ก่อน regression (dropna หลัง align) - ต้องรายงาน p-value ของ beta เพื่อตรวจนัยสำคัญ - residuals ต้องมี index เดียวกับ input (หลัง dropna) - raise ValueError ถ้า y และ x มี overlap น้อยกว่า 30 observations
Libraries: statsmodels, pandas, numpy

```
import statsmodels.api as sm
import numpy as np
import pandas as pd

def ols_hedge_ratio(
    y: pd.Series,
    x: pd.Series,
    add_constant: bool = True,
) -> dict:
    """
    คำนวณ OLS hedge ratio สำหรับ pair trading

    Parameters
    -----
    y : pd.Series  log prices ของ asset A
    x : pd.Series  log prices ของ asset B
    add_constant : bool  เพิ่ม intercept (แนะนำ True)

    Returns
    -----
    dict: beta, alpha, r_squared, p_value_beta, residuals
    """
    # Align และ drop NaN
    df = pd.concat({"y": y, "x": x}, axis=1).dropna()
    if len(df) < 30:
        raise ValueError(
            f"ต้องการอย่างน้อย 30 observations แต่มีเพียง {len(df)}"
        )

    Y = df["y"].values
    X = df["x"].values
    if add_constant:
        X = sm.add_constant(X)

    model = sm.OLS(Y, X).fit()

    if add_constant:
```

```

        alpha = float(model.params[0])
        beta = float(model.params[1])
        p_val = float(model.pvalues[1])
    else:
        alpha = 0.0
        beta = float(model.params[0])
        p_val = float(model.pvalues[0])

    residuals = pd.Series(model.resid, index=df.index, name="residuals")

    return {
        "beta": beta,
        "alpha": alpha,
        "r_squared": float(model.rsquared),
        "p_value_beta": p_val,
        "residuals": residuals,
    }

# ทดสอบ
np.random.seed(42)
n = 200
x_prices = pd.Series(
    np.log(100 + np.cumsum(np.random.normal(0, 1, n))),
    index=pd.date_range("2023-01-01", periods=n)
)
# noise ต้องเล็กกว่าช่วงที่ x แกว่ง (log ของ random walk แกว่งแค่ ~0.13)
# ไม่งั้น OLS ประมาณ beta เพิ่มขึ้นจาก 0.80 อย่างเห็นได้ชัด
y_prices = 0.8 * x_prices + pd.Series(
    np.random.normal(0, 0.002, n), index=x_prices.index
)

result = ols_hedge_ratio(y_prices, x_prices)
print(f"Beta: {result['beta']:.4f} (expect ≈ 0.80)")
print(f"R²: {result['r_squared']:.4f}")
print(f"p-value: {result['p_value_beta']:.2e}")

```

► OUTPUT

```

Beta:      0.8048 (expect ≈ 0.80)
R²:        0.9968
p-value:    1.58e-248

```

29.2 Prompt Template สำหรับ ADF Test

Task: เขียนฟังก์ชันทดสอบ cointegration ระหว่าง 2 series ด้วย ADF test บน residuals จาก OLS regression Input: - residuals: pd.Series, residuals จาก hedge ratio regression - significance: float = 0.05 (significance level)
 Output: - dict: {'is_cointegrated': bool, 'adf_stat': float, 'p_value': float, 'critical_values': dict}
 Constraints: - ใช้ statsmodels.tsa.stattools.adfuller - autolag='AIC' เพื่อเลือก lag อัตโนมัติ - is_cointegrated = True ถ้า p_value < significance เท่านั้น (ไม่ใช่แค่ adf_stat < critical_value เพราะ critical_value ขึ้นกับ sample size) - รายงาน critical values ที่ 1%, 5%, 10% Libraries: statsmodels

```

from statsmodels.tsa.stattools import adfuller

def test_cointegration(
    residuals: pd.Series,
    significance: float = 0.05,
) -> dict:
    """
    ทดสอบ cointegration ด้วย ADF test บน residuals
    """

```

```

H0: residuals มี unit root (ไม่ cointegrate)
H1: residuals เป็น stationary (cointegrate)

Parameters
-----
residuals : pd.Series   residuals จาก OLS hedge ratio
significance : float    significance level (default 0.05)

Returns
-----
dict: is_cointegrated, adf_stat, p_value, critical_values
"""

clean = residuals.dropna()
if len(clean) < 20:
    raise ValueError("ต้องการอย่างน้อย 20 observations สำหรับ ADF test")

adf_result = adfuller(clean, autolag="AIC")

adf_stat      = float(adf_result[0])
p_value       = float(adf_result[1])
critical_vals = adf_result[4]    # dict: {'1%': ..., '5%': ..., '10%': ...}

return {
    "is_cointegrated": p_value < significance,
    "adf_stat":      adf_stat,
    "p_value":      p_value,
    "critical_values": {k: float(v) for k, v in critical_vals.items()},
}

# ทดสอบ
coint_result = test_cointegration(result["residuals"])
print(f"Cointegrated: {coint_result['is_cointegrated']}")
print(f"ADF stat:      {coint_result['adf_stat']:.4f}")
print(f"p-value:      {coint_result['p_value']:.4f}")
print(f"Critical (5%): {coint_result['critical_values']['5%']:.4f}")

```

► OUTPUT

```

Cointegrated: True
ADF stat:      -14.8787
p-value:      0.0000
Critical (5%): -2.8762

```

29.3 Prompt Template สำหรับ Rolling Z-Score (pandas)

Task: เขียน vectorized rolling z-score ที่เร็วและถูกต้อง Input: - series: pd.Series - window: int, backward-looking window - min_periods: int = None (default = window) Output: - pd.Series, z-score, NaN สำหรับ observations ที่ข้อมูลไม่พอ Constraints: - ห้ามใช้ loop – ต้องใช้ pandas vectorized operations - ห้าม center=True - std ใช้ ddof=1 (unbiased) - ถ้า std = 0 ใน window ใด → return 0 (ไม่ใช่ NaN หรือ inf) - efficient: ไม่คำนวณ rolling ซ้ำสองครั้ง Libraries: pandas เท่านั้น (ไม่ใช่ scipy)

```

def rolling_zscore(
    series: pd.Series,
    window: int,
    min_periods: int = None,
) -> pd.Series:
    """
    Backward-looking rolling z-score

```

```
z_t = (x_t - mean(x_{t-window+1}...x_t)) / std(x_{t-window+1}...x_t)
```

Parameters

series : pd.Series

window : int lookback window (backward-looking)

min_periods : int minimum observations required (default = window)

Returns

pd.Series, same index as input

"""

if min_periods is None:

min_periods = window

roll = series.rolling(window=window, min_periods=min_periods)

roll_mean = roll.mean()

roll_std = roll.std(ddof=1)

ป้องกัน division by zero: ถ้า std = 0 → zscore = 0

zscore = (series - roll_mean) / roll_std.replace(0, float("nan"))

return zscore.fillna(0).where(roll_mean.notna(), other=float("nan"))

ทดสอบ

```
ts = pd.Series([10, 11, 10, 12, 10, 11, 13, 10, 9, 11],  
               index=pd.date_range("2024-01-01", periods=10))
```

```
z = rolling_zscore(ts, window=5)
```

```
print(z.round(3))
```

► OUTPUT

2024-01-01 NaN

2024-01-02 NaN

2024-01-03 NaN

2024-01-04 NaN

2024-01-05 -0.671

2024-01-06 0.239

2024-01-07 1.381

2024-01-08 -0.920

2024-01-09 -1.055

2024-01-10 0.135

Freq: D, dtype: float64

29.4 Prompt Template สำหรับ scipy.optimize

Task: เขียนฟังก์ชัน optimize portfolio weights เพื่อ maximize Sharpe ratio Input: - returns: pd.DataFrame, daily returns, columns = assets - constraints: dict = {'min_weight': 0.0, 'max_weight': 1.0, 'sum_to_one': True} Output: - pd.Series, optimal weights indexed by asset names - float, expected Sharpe ratio ของ optimal portfolio Constraints: - ใช้ scipy.optimize.minimize ด้วย method='SLSQP' - objective function = negative Sharpe (เพื่อ minimize) - annualize ด้วย sqrt(252) - รวม try/except สำหรับ optimization failure - seed weights = equal weight - ตรวจสอบ convergence — raise Warning ถ้าไม่ converge Libraries: scipy.optimize, numpy, pandas

📌อ่านว่า: การเรียก optimizer ทุกตัวในโลก ไม่ว่าไลบรารีไหน ประกอบด้วยของ 4 อย่างเสมอ — พอจำได้แล้วโค้ด 72 บรรทัดข้างล่างจะเหลือแค่ "ทำให้เจอบว่าของ 4 อย่างนี้อยู่ตรงไหน": ① สิ่งที่ยากให้น้อยที่สุด (objective) · ② จุดเริ่มเดา (w0) · ③ กรอบที่ห้ามออกนอก (bounds — รายตัว) · ④ เงื่อนไขที่ต้องเป็นจริง (constraints — ข้ามตัว) · ที่เหลือคือ docstring กับการจัด DataFrame ให้สวย

```
from scipy.optimize import minimize  
import numpy as np  
import pandas as pd
```

```

import warnings

def optimize_sharpe(
    returns: pd.DataFrame,
    constraints: dict = None,
) -> tuple:
    """
    หา portfolio weights ที่ maximize Sharpe ratio

    Parameters
    -----
    returns : pd.DataFrame    daily returns
    constraints : dict        min_weight, max_weight, sum_to_one

    Returns
    -----
    (pd.Series weights, float sharpe)
    """
    if constraints is None:
        constraints = {"min_weight": 0.0, "max_weight": 1.0, "sum_to_one": True}

    n = len(returns.columns)
    mu = returns.mean().values * 252
    cov = returns.cov().values * 252

    def neg_sharpe(w: np.ndarray) -> float:
        port_ret = w @ mu
        port_vol = np.sqrt(w @ cov @ w)
        if port_vol < 1e-10:
            return 0.0
        return -(port_ret / port_vol)

    bounds = [(constraints["min_weight"],
                  constraints["max_weight"])] * n

    opt_constraints = []
    if constraints.get("sum_to_one", True):
        opt_constraints.append(
            {"type": "eq", "fun": lambda w: w.sum() - 1.0}
        )

    w0 = np.ones(n) / n # equal weight seed

    result = minimize(
        neg_sharpe,
        w0,
        method="SLSQP",
        bounds=bounds,
        constraints=opt_constraints,
        options={"maxiter": 1000, "ftol": 1e-9},
    )

    if not result.success:
        warnings.warn(
            f"Optimization did not converge: {result.message}",
            RuntimeWarning,
        )

    optimal_weights = pd.Series(result.x, index=returns.columns)
    optimal_sharpe = -result.fun
    return optimal_weights, optimal_sharpe

```

```
# ทดสอบ
np.random.seed(42)
n_days = 252
assets = ["BTC", "ETH", "SOL"]
ret_data = pd.DataFrame(
    np.random.multivariate_normal(
        mean=[0.0005, 0.0007, 0.0010],
        cov=[[0.0002, 0.00015, 0.0001],
              [0.00015, 0.0003, 0.00012],
              [0.0001, 0.00012, 0.0005]],
        size=n_days,
    ),
    columns=assets,
)

weights, sharpe = optimize_sharpe(ret_data)
print("Optimal weights:")
for asset, w in weights.items():
    print(f" {asset}: {w:.1%}")
print(f"Expected Sharpe: {sharpe:.2f}")
```

► OUTPUT

```
Optimal weights:
BTC: 0.0%
ETH: 0.0%
SOL: 100.0%
Expected Sharpe: 1.01
```

🤖 ของ 4 อย่างในโค้ดนี้อยู่ตรงไหน

```
def neg_sharpe(w):
    port_ret = w @ mu
    port_vol = np.sqrt(w @ cov @ w)
    return -(port_ret / port_vol)

w0 = np.ones(n) / n
bounds = [(0.0, 1.0)] * n
opt_constraints = [{"type": "eq",
                    "fun": lambda w: w.sum() - 1.0}]
```

① สิ่งที่ยากให้น้อยที่สุด
ผลตอบแทนพอร์ต
ความผันผวนพอร์ต
คิดลบ = พลิก max เป็น min

② จุดเริ่มเดา = ถือเท่ากันทุกตัว
③ กรอบรายตัว: 0-100%
④ เงื่อนไขข้ามตัว:
น้ำหนักรวม = 1

1. $w @ \mu$ — @ คือ **dot product** เขียนสั้น ๆ ของ $(w[0]*\mu[0] + w[1]*\mu[1] + \dots)$ · อ่านเป็นภาษาคนว่า "ถ่วงน้ำหนักผลตอบแทนของแต่ละตัวตามสัดส่วนที่ถือ"
2. $\text{np.sqrt}(w @ \text{cov} @ w)$ — สูตรความผันผวนของพอร์ต · เหตุที่ **ไม่ใช่** การถ่วงน้ำหนัก sd ธรรมดา เพราะสินทรัพย์ไม่ได้ขึ้นลงพร้อมกันเป๊ะ ๆ · ตัวนอกแนวทแยงของ cov คือส่วนที่บอกว่ามันไปด้วยกันแค่ไหน — และเป็นที่มาของคำว่า "กระจายความเสี่ยง" ทั้งหมด
3. $\text{return } -(\dots)$ — **scipy มีแต่ minimize ไม่มี maximize** · อยากได้ Sharpe สูงสุดจึงใส่ลบแล้วหาค่าต่ำสุดแทน · ตอนคืนค่าจึงต้องกลับเครื่องหมายอีกรอบ ($\text{optimal_sharpe} = -\text{result.fun}$) ไม่งั้นจะรายงาน Sharpe ติดลบทั้งที่พอร์ตดี
4. **bounds กับ constraints ต่างกันตรงไหน** — bounds คุณทีละตัวแยกกัน ("BTC ต้องอยู่ระหว่าง 0-100%") · constraints คุณความสัมพันธ์ข้ามตัว ("รวมกันต้องได้ 1") · optimizer จัดการคนละวิธี bounds ถูกบังคับตลอดทาง ส่วน constraints ถูกไล่เข้าหาทีละรอบ · เขียนสลับที่กันจะช้าลงมากหรือไม่ converge เลย
5. $\{"type": "eq", "fun": \text{lambda } w: w.\text{sum}() - 1.0\}$ — รูปแบบที่ scipy กำหนด: เขียนเงื่อนไขให้อยู่ในรูป **"ฟังก์ชันนี้ต้องเท่ากับศูนย์"** เสมอ · อยากได้ $\text{sum}(w) = 1$ จึงต้องเขียนเป็น $\text{sum}(w) - 1$ · ถ้าเป็น "ineq" ความหมายคือ "ต้อง ≥ 0 "

อ่านผลลัพธ์ให้ออก — optimizer ตอบว่า "ทุ่มหมดหน้าดัก SOL"

ผลที่ได้คือ **BTC 0% · ETH 0% · SOL 100%** ทั้งที่โจทย์ให้สินทรัพย์มาสามตัวเพื่อกระจายความเสี่ยง · นี่คือสิ่งที่เรียกว่า **corner solution** และเป็นพฤติกรรมปกติของ mean-variance optimizer ไม่ใช่ bug ของโค้ด

โค้ดทำถูกทุกบรรทัด · optimizer ก็หาคำตอบเจอจริง · ปัญหาอยู่ที่**ของที่ป้อนเข้าไป** — และนี่คือกับดักที่ทำให้ backtest จำนวนมากในโลกไร้ค่า

ลองเปิดดูตัวเลขที่ optimizer ใช้ตัดสินใจ เทียบกับค่าจริงที่เราใช้สร้างข้อมูล (เรารู้เพราะเราเขียนเอง — ในตลาดจริงไม่มีใครรู้):

```
# optimizer เห็นอะไร vs ความจริงเป็นอย่างไร
true_mu_annual = np.array([0.0005, 0.0007, 0.0010]) * 252 # ค่าที่เราใส่ตอนสร้าง
est_mu_annual = ret_data.mean().values * 252 # ค่าที่วัดได้จาก 252 วัน
true_sd_annual = np.sqrt(np.array([0.0002, 0.0003, 0.0005]) * 252)
est_sd_annual = ret_data.std().values * np.sqrt(252)

# ค่าเฉลี่ยจาก n ปี มีค่าคลาดเคลื่อนมาตรฐาน = sd / sqrt(n) → 1 ปี ก็คือ sd เต็ม ๆ
print(f"{'':5}{'ผลตอบแทนจริง':>14}{'ที่วัดได้':>12}{'±คลาดเคลื่อน':>14}")
for i, a in enumerate(assets):
    print(f"{a:5}{true_mu_annual[i]:>+13.1%}{est_mu_annual[i]:>+12.1%}"
          f"{true_sd_annual[i]:>+14.1%}")

print(f"\n{'':5}{'ความผันผวนจริง':>15}{'ที่วัดได้':>12}")
for i, a in enumerate(assets):
    print(f"{a:5}{true_sd_annual[i]:>+14.1%}{est_sd_annual[i]:>+12.1%}")

# ต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงแม่นยำระดับ 1/3 ของตัวมันเอง
years_needed = (3 * true_sd_annual / true_mu_annual) ** 2
print(f"\nต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงจะพอเชื่อได้:")
for i, a in enumerate(assets):
    print(f" {a}: {years_needed[i]:.0f} ปี")
```

► OUTPUT

ผลตอบแทนจริง	ที่วัดได้	±คลาดเคลื่อน	
BTC	+12.6%	-22.5%	+22.4%
ETH	+17.6%	-47.5%	+27.5%
SOL	+25.2%	+33.4%	+35.5%

ความผันผวนจริง	ที่วัดได้	
BTC	22.4%	21.3%
ETH	27.5%	26.2%
SOL	35.5%	33.2%

ต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงจะพอเชื่อได้:

BTC: 29 ปี
ETH: 22 ปี
SOL: 18 ปี

ตัวเลขที่ควรจำไปทั้งชีวิต

ความจริงคือ**ทั้งสามตัวกำไรหมด** (+12.6% / +17.6% / +25.2%) · แต่ข้อมูล 252 วันที่สุ่มออกมาบอกว่า BTC ขาดทุน 22.5% และ ETH ขาดทุน 47.5% — **ผิดกระทั้งเครื่องหมาย** · optimizer เห็นแบบนั้นก็ทำงานของมันอย่างซื่อสัตย์: ตัดสองตัวที่ "ขาดทุน" ทั้ง เหลือตัวเดียว

ดูคอลัมน์ ±คลาดเคลื่อนให้ดี · ค่าคลาดเคลื่อนของค่าเฉลี่ยรายปีจากข้อมูล 1 ปี = ความผันผวนรายปีเต็ม ๆ ของ BTC คือ ±22.4% ในขณะที่ค่าจริงคือ +12.6% — **ความคลาดเคลื่อนใหญ่กว่าสิ่งที่จะวัด** · จึงต้องรอถึง 29 ปีกว่าจะแม่นยำพอ

ตรงข้ามกับตารางที่สอง: **ความผันผวนวัดได้แม่นยำมาก** (22.4 vs 21.3) จากข้อมูลชุดเดียวกัน · สาเหตุคือ sd ประมาณจาก "ขนาดของการแกว่ง" ซึ่งมีให้เก็บทุกแห่ง ส่วนค่าเฉลี่ยประมาณจาก "ระยะทางสุทธิจากต้นถึงปลาย" ซึ่งมีแค่ค่าเดียวไม่ว่าจะเก็บกี่แค่ไหน — **เก็บข้อมูลรายนาทีก็ไม่ช่วยให้รู้ค่าเฉลี่ยแม่นยำขึ้น มีแต่ต้องรอนานขึ้นเท่านั้น**

ข้อสรุปไม่ใช่ "อย่าใช้ optimizer" แต่คือ อย่าบ่อนค่าเฉลี่ยที่ประมาณจากอดีตเข้าไปตรง ๆ . ทางออกที่ใช้กันจริงมีสามแนว:

1. เอาค่าเฉลี่ยออกจากสมการ — ใช้ minimum-variance หรือ risk-parity แทน maximum-Sharpe . ทั้งคู่ใช้แค่ `cov` ซึ่งเป็นตัวเลขที่เชื่อได้จริงตามตารางที่สอง (แบบฝึกหัดข้างล่างคือข้อนี้พอดี)
2. บีบกรอบให้แคบลง — เปลี่ยน `max_weight` จาก 1.0 เป็นราว 0.4 . เป็นการยอมรับตรง ๆ ว่า "ฉันไม่เชื่อค่าเฉลี่ยของตัวเองพอที่จะทุ่มหมดหน้าตัก" . หยาบแต่ได้ผลดีอย่างน่าประหลาด
3. ดึงค่าประมาณเข้าหาค่ากลาง (shrinkage / Black-Litterman) — แทนที่จะเชื่อ `-47.5%` ของ ETH ก็ผสมกับค่าเฉลี่ยรวมของทั้งกลุ่ม . เป็นวิธีที่กองทุนใช้จริง

มีผลการศึกษาลาสสิกที่ยืนยันไปในทางเดียวกันว่า พอร์ตถือเท่ากันทุกตัว ($1/N$ — ซึ่งก็คือ `w0` ที่โค้ดนี้ใช้เป็นแค่จุดเริ่มต้น) เอาชนะ mean-variance optimizer นอกกลุ่มตัวอย่างได้บ่อยครั้ง . ไม่ใช่เพราะ $1/N$ ฉลาด แต่เพราะมันไม่ต้องประมาณค่าเฉลี่ยเลย

► แบบฝึกหัด: เขียน prompt สำหรับ linear programming หา minimum variance portfolio พร้อม sector constraints

บทที่ 30: Full Strategy with AI

แก่นของบทนี้

ขั้นตอนจริงของการ build strategy ด้วย AI: ไม่ใช่ prompt ครึ่งเดียวแล้วได้โค้ดสมบูรณ์ — แต่เป็นกระบวนการ **3 รอบ iterate** ที่แต่ละรอบ แก้ปัญหาที่พบจากการ audit รอบก่อน

30.1 Overview — Volatility Mean-Reversion Strategy

แนวคิด: เมื่อ realized volatility สูงผิดปกติ (z-score > threshold) vol มักจะ mean-revert กลับลงมา ในช่วง high vol มักเป็นช่วง sell-off หนัก → หลัง vol spike ราคาขึ้น → **long underlying เมื่อ vol spike**

Entry signal = 1[vol_zscore_t > z_{entry}]
Exit signal = 1[vol_zscore_t < z_{exit}]

DESIGN → ROUND 1 → AUDIT → ROUND 2 → AUDIT → ROUND 3 → FINAL (บท 26) (prompt) (บท 28) (แก้ bug) (verify) (costs) (deploy-ready)

30.2 Round 1 — Generate โค้ดแรก

ROUND 1 PROMPT

Task: เขียน full backtest สำหรับ volatility mean-reversion strategy Components ที่ต้องการ: 1. compute_vol_zscore(prices, vol_window, zscore_window) → DataFrame 2. generate_signal(vol_zscore, entry_z, exit_z) → pd.Series ('LONG'/'EXIT'/'HOLD') 3. run_backtest(prices, signal, fee_per_trade) → DataFrame ที่มี equity curve Input (run_backtest): - prices: pd.Series, daily close prices - signal: pd.Series, output จาก generate_signal - fee_per_trade: float = 0.001 Output (run_backtest): - pd.DataFrame: date | price | signal | position | daily_pnl | equity Constraints: - signal ณ วัน t ต้องใช้แค่ข้อมูลถึง t-1 (no lookahead) - position เข้าที่ราคา open วันถัดไป (simulate ด้วย price.shift(-1) ของ signal day) - หัก fee ทุกครั้งที่ position เปลี่ยน - equity เริ่มที่ 100 (% ของ initial capital) - return NaN ช่วงต้นที่ไม่มีข้อมูลพอ Libraries: pandas, numpy

📌**อ่านว่า:** โค้ดที่จะได้ยาว 59 บรรทัด — **อย่าอ่านไล่ทีละบรรทัดจากบนลงล่าง** นั่นคือวิธีอ่านที่ทำให้หลง · วิธีที่เร็วกว่ามากคืออ่าน **3 signature** ก่อน (compute_vol_zscore → generate_signal → run_backtest) ให้เห็นว่าของออกจากตัวหนึ่งไปเข้าตัวไหนเป็นสายพานเส้นเดียว: ราคา → z-score ของความผันผวน → สัญญาณข้อความ → equity curve · แล้วค่อยเจาะเข้าไปในตัวที่มีตรรกะเงินจริง คือ run_backtest เท่านั้น · อีกสองตัวเป็นการคำนวณตรงไปตรงมาที่ไล่ทันอยู่แล้ว

Round 1: โค้ดที่ AI สร้าง (อาจมี bug บางข้อ)
import numpy as np
import pandas as pd

def compute_vol_zscore(
 prices: pd.Series,
 vol_window: int = 21,
 zscore_window: int = 63,
) -> pd.DataFrame:
 log_ret = np.log(prices / prices.shift(1))
 ann_vol = (
 log_ret.rolling(vol_window, min_periods=vol_window).std()
 * np.sqrt(252) * 100
)

```

roll_mean = ann_vol.rolling(zscore_window, min_periods=zscore_window).mean()
roll_std = ann_vol.rolling(zscore_window, min_periods=zscore_window).std()
vol_zscore = (ann_vol - roll_mean) / roll_std
return pd.DataFrame({"log_ret": log_ret, "ann_vol": ann_vol,
                    "vol_zscore": vol_zscore})

def generate_signal(
    vol_zscore: pd.Series,
    entry_z: float = 1.5,
    exit_z: float = 0.5,
) -> pd.Series:
    sig = pd.Series("HOLD", index=vol_zscore.index)
    sig[vol_zscore > entry_z] = "LONG"
    sig[vol_zscore.abs() < exit_z] = "EXIT"
    sig[vol_zscore.isna()] = "HOLD"
    return sig

def run_backtest(
    prices: pd.Series,
    signal: pd.Series,
    fee_per_trade: float = 0.001,
) -> pd.DataFrame:
    """
    Backtest engine สำหรับ long-only strategy
    entry: open ของวันถัดไปหลัง signal
    """
    df = pd.DataFrame({"price": prices, "signal": signal}).copy()

    # position: 1 = long, 0 = flat
    # ใช้ signal วัน t เพื่อ set position วัน t+1
    position = pd.Series(0, index=df.index)
    pos = 0
    for i in range(1, len(df)):
        prev_sig = df["signal"].iloc[i - 1]
        if prev_sig == "LONG":
            pos = 1
        elif prev_sig == "EXIT":
            pos = 0
        position.iloc[i] = pos

    df["position"] = position

    # Daily P&L
    log_ret = np.log(df["price"] / df["price"].shift(1))
    df["daily_pnl"] = df["position"].shift(1) * log_ret  # ← shift(1): ถูกต้อง

    # Transaction costs
    turnover = df["position"].diff().abs().fillna(0)
    df["daily_pnl"] -= turnover * fee_per_trade

    # Equity curve (start at 100)
    df["equity"] = 100 * np.exp(df["daily_pnl"].fillna(0).cumsum())
    return df

```

🤖 หัวใจของ backtest ทั้งตัวอยู่ที่ 4 บรรทัดนี้

```

log_ret = np.log(df["price"] / df["price"].shift(1))  # ① ผลตอบแทนของ "ตลาด"
df["daily_pnl"] = df["position"].shift(1) * log_ret  # ② ของ "เรา"

```

```
turnover = df["position"].diff().abs().fillna(0) # ③ วันที่มีการซื้อขาย
df["daily_pnl"] -= turnover * fee_per_trade

df["equity"] = 100 * np.exp(df["daily_pnl"].fillna(0).cumsum()) # ④ ทบต้น
```

1. $\text{np.log}(p / p.\text{shift}(1))$ — ผลตอบแทนแบบ log ของวันนั้น. ที่ใช้ log ไม่ใช่ `pct_change()` เพราะบรรทัด ④ ต้องการ**บวกกันได้**: ผลตอบแทน log ของหลายวันเอามาบวกกันตรง ๆ แล้วได้ผลตอบแทนรวม ส่วนเปอร์เซ็นต์ธรรมดาต้องคูณกัน
2. $\text{position}.\text{shift}(1) * \text{log_ret}$ — **บรรทัดที่กันโกงทั้งเล่มอยู่ตรงนี้**. `log_ret` ของวันนี้คือระยะทางจากปิดเมื่อวานถึงปิดวันนี้ ดังนั้นคนที่จะได้เงินก่อนนั้นต้อง**ถือของอยู่แล้วตั้งแต่เมื่อวาน** — ซึ่งคือ $\text{position}.\text{shift}(1)$. ถ้าลืม `.shift(1)` จะกลายเป็น "รู้ราคาปิดวันนี้ก่อนแล้วค่อยตัดสินใจ" = lookahead แบบคลาสสิก
3. $\text{position}.\text{diff}().\text{abs}()$ — `diff()` คือ "วันนี้ต่างจากเมื่อวานเท่าไร" $0 \rightarrow 1$ ได้ $+1$, $1 \rightarrow 0$ ได้ -1 , ไม่เปลี่ยนได้ 0 . ใส่ `.abs()` เพราะ**ทั้งเข้าและออกเสียค่าธรรมเนียมเท่ากัน** ไม่มีขาไหนได้เงินคืน. ผลคือ Series ที่เป็น 1 เฉพาะวันที่มีการซื้อขายจริง คุณค่าธรรมเนียมแล้วหักออกได้เลย
4. $100 * \text{np.exp}(\text{pnl}.\text{cumsum}())$ — `cumsum()` สะสมผลตอบแทน log ทั้งหมดตั้งแต่วันแรก แล้ว `exp()` แปลงกลับเป็นตัวคูณ. เริ่มที่ 100 หมายความว่าตัวเลขบน equity curve อ่านเป็น**เปอร์เซ็นต์ของทุนตั้งต้น**ได้ทันที (120 = กำไร 20%)

สังเกตว่าทั้งสี่บรรทัด**ไม่มีลูป** — คิดทั้งคอลัมน์พร้อมกัน. ลูป for ตัวเดียวในฟังก์ชันนี้เกี่ยวกับการแปลงสัญญาณข้อความเป็น position เพราะ position วันนั้นขึ้นกับ position เมื่อวาน (สถานะที่ "ค้าง" ต่อกันไปเรื่อย ๆ) ซึ่งเป็นงานประเภทที่ vectorize ตรง ๆ ไม่ได้

30.3 Round 1 Audit — หา Bug ก่อน Round 2

```
# Audit Round 1: ตรวจสอบทุก checkpoint

np.random.seed(7)
n = 500
prices_sim = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0003, 0.015, n))),
    index=pd.date_range("2022-01-01", periods=n)
)

vol_df = compute_vol_zscore(prices_sim, vol_window=21, zscore_window=63)
signals = generate_signal(vol_df["vol_zscore"], entry_z=1.5, exit_z=0.5)
result = run_backtest(prices_sim, signals, fee_per_trade=0.001)

print("=== Round 1 Audit ===")
print(f"1. NaN ใน vol_zscore (ช่วงต้น): {vol_df['vol_zscore'].isna().sum()} rows")
print(f"2. Signal distribution:\n{signals.value_counts()}")
print(f"3. Position 0/1 only: {set(result['position'].unique())}")
print(f"4. First non-NaN equity row: {result['equity'].first_valid_index()}")
print(f"5. Final equity: {result['equity'].iloc[-1]:.2f}")

# Embargo test
print("\n=== Embargo Test ===")
def run_with_prices(p):
    vdf = compute_vol_zscore(p)
    sig = generate_signal(vdf["vol_zscore"])
    return vdf # DataFrame with vol_zscore column

# ตรวจสอบ lookahead โดยตรง: zscore ณ วัน 200 ต้องไม่เปลี่ยนเมื่อตัดข้อมูลหลังวัน 200
vdf_full = compute_vol_zscore(prices_sim)
vdf_cut = compute_vol_zscore(prices_sim.iloc[:250])

diff_at_200 = abs(
    vdf_full["vol_zscore"].iloc[200] - vdf_cut["vol_zscore"].iloc[200]
)

print(f"Vol zscore diff at day 200 (full vs cut): {diff_at_200:.8f}")
print("PASS" if diff_at_200 < 1e-10 else "FAIL: lookahead bias detected!")
```

► OUTPUT

```
=== Round 1 Audit ===
1. NaN ใน vol_zscore (ช่วงต้น): 83 rows
2. Signal distribution:
HOLD    322
EXIT    111
LONG     67
Name: count, dtype: int64
3. Position 0/1 only: {np.int64(0), np.int64(1)}
4. First non-NaN equity row: 2022-01-01 00:00:00
5. Final equity: 84.03
=== Embargo Test ===
Vol zscore diff at day 200 (full vs cut): 0.00000000
PASS
```

Bug ที่พบใน Round 1

Equity เริ่มต้นนับแม้ช่วงที่มี NaN — `first_valid_index` คืน 2022-01-01 ทั้งที่ position ยังเป็น 0 อยู่ 83 วัน ไม่ใช่ bug ร้ายแรง แต่ต้องระวังเมื่อคำนวณ Sharpe ratio (อย่านับ NaN period เข้าไป)

ข้อที่ audit ช่างบ่นยังจับไม่ได้ — และวิธีจับมัน

audit ชุดแรกตรวจสอบทุกอย่างที่ prompt เขียนไว้เป็นข้อ ๆ ยกเว้นข้อเดียว: *"position เข้าที่ราคา open วันถัดไป"*. ข้อนี้ไม่มี checkpoint ไหนตรวจสอบเลย เพราะมันไม่ใช่คำถามว่า "ผลลัพธ์หน้าตาถูกไหม" แต่เป็นคำถามว่า **"หน่วงกี่แท่ง"** — ซึ่งดูจากตัวเลขสรุปไม่ออก ต้องไล่ทีละแถว · เครื่องมือที่ตอบคำถามแบบนี้ได้ใน 5 บรรทัดคือ **ข้อมูลของเล่นที่คำตอบรู้ล่วงหน้า**: ให้ราคาขึ้นวันละ 1% เป๊ะ ๆ แล้วยิงสัญญาณครั้งเดียว วันไหนที่เงินเริ่มไหลคือคำตอบ

```
# ข้อมูลของเล่น: ขึ้นวันละ 1% คงที่ + สัญญาณครั้งเดียว → นับหน่วงได้ด้วยตา
idx_toy = pd.date_range("2022-01-01", periods=6)
prices_toy = pd.Series(100 * 1.01 ** np.arange(6), index=idx_toy)
sig_toy = pd.Series("HOLD", index=idx_toy)
sig_toy.iloc[1] = "LONG"

r_toy = run_backtest(prices_toy, sig_toy, fee_per_trade=0.0)
print(r_toy[["price", "signal", "position", "daily_pnl"]].round(5).to_string())

sig_day = idx_toy.get_loc(sig_toy[sig_toy == "LONG"].index[0])
pnl_day = idx_toy.get_loc(r_toy.index[r_toy["daily_pnl"].fillna(0) != 0][0])
print(f"\nสัญญาณแท่งที่ {sig_day} · เริ่มได้กำไรแท่งที่ {pnl_day} "
      f"· หน่วง {pnl_day - sig_day} แท่ง")
```

► OUTPUT

	price	signal	position	daily_pnl
2022-01-01	100.00000	HOLD	0	NaN
2022-01-02	101.00000	LONG	0	0.00000
2022-01-03	102.01000	HOLD	1	0.00000
2022-01-04	103.03010	HOLD	1	0.00995
2022-01-05	104.06040	HOLD	1	0.00995
2022-01-06	105.10101	HOLD	1	0.00995

สัญญาณแท่งที่ 1 · เริ่มได้กำไรแท่งที่ 3 · หน่วง 2 แท่ง

Bug ที่ 2 — หน่วงเกินไป 1 แท่ง

prompt สั่งไว้ว่าหน่วง **1 แท่ง** (สัญญาณวัน t → เข้าที่ open วัน $t+1$ → ได้ผลตอบแทนของวัน $t+1$) แต่โค้ดจริงหน่วง **2 แท่ง** · สาเหตุคือมันหน่วงสองรอบซ้อนโดยไม่ตั้งใจ:

1. ลูป for อ่าน `signal.iloc[i-1]` มาตั้ง `position.iloc[i]` — หน่วงรอบที่ ①

2. `df["position"].shift(1) * log_ret` — หน่วงรอบที่ ②

แต่ละรอบแยกกันถูกต้องทั้งคู่ — นี่คือการทดสอบที่มั่นคงปลอดภัย · ทั้งสองบรรทัดอยู่ห่างกัน 15 บรรทัด และต่างก็ตอบโจทย์ "กันมองอนาคต" คนละข้อ

bug นี้ไม่อันตราย — หน่วงเกินคือ *อนุรักษนิยมเกิน* ผลลัพธ์จะแย่กว่าความจริง ตรงข้ามกับ lookahead ที่ทำให้ดูดีเกินจริง · แต่ก็ยัง**ผิดจากที่สั่ง** และในกลยุทธ์ที่สัญญาณหมดอายุเร็ว (เช่น mean reversion รายชั่วโมง) การเข้าไป 1 แท่งก็กำไรหายทั้งกลยุทธ์ได้

● ทำไม embargo test ที่หนังสือเขียนไว้จับข้อนี้ไม่ได้

เพราะ embargo test ตรวจจนละเอียด · มันถามว่า "ตัดข้อมูลอนาคตทิ้งแล้ว ค่าที่คำนวณไว้เปลี่ยนไหม" — ตอบว่าไม่เปลี่ยน (PASS) และก็ควรจะเป็นแบบนั้น เพราะการหน่วงเกินยิ่งทำให้ไม่มีทางแตะข้อมูลอนาคตอยู่แล้ว

บทเรียนที่ใช้ต่อได้: **test แต่ละตัวมีขอบเขตที่มันมองเห็นของมัน** · embargo test เห็นเฉพาะ "ตะเอนาคตหรือเปล่า" · การจะเห็น "หน่วงกี่แท่ง" ต้องใช้ test อีกชนิดคือ **impulse response** — ยิ่งสัญญาณเดียวบนข้อมูลที่นิ่งสนิท แล้วดูว่าระบบตอบสนองเมื่อไหร่ · เป็นเทคนิคเดียวกับที่วิศวกรใช้วัด latency ของระบบควบคุม และใช้ได้กับทุก pipeline ที่มี `shift` ซ้อนกันหลายชั้น

30.4 Round 2 — เพิ่ม Performance Metrics และแก้ Sharpe Calculation

ROUND 2 PROMPT

Task: เพิ่มฟังก์ชัน `compute_metrics` บน output ของ `run_backtest` ปัญหาที่พบใน Round 1: - Sharpe ratio คำนวณรวม period ที่ `position = 0` และ `daily_pnl = 0` ทำให้ Sharpe ต่ำเกิน - ต้องการ max drawdown, win rate, และ profit factor ด้วย Input: - `result_df`: `pd.DataFrame` output จาก `run_backtest` - `start_equity`: float = 100 Output: - dict: `sharpe`, `annual_return`, `max_drawdown`, `win_rate`, `profit_factor`, `n_trades`, `avg_holding_days` Constraints: - คำนวณ Sharpe เฉพาะวันที่ `position != 0` (active days only) - `max_drawdown` = max peak-to-trough ของ equity curve - `win_rate` = fraction of trades with positive net pnl - `profit_factor` = `sum(wins) / abs(sum(losses))` - return inf ถ้าไม่มี losses - `n_trades` = จำนวนครั้งที่ `position` เปลี่ยนจาก 0 เป็น 1 Libraries: `numpy`, `pandas`

📖**อ่านว่า**: 83 บรรทัดข้างล่างคำนวณตัวเลข 7 ตัว แต่แบ่งเป็น **2 กลุ่มที่คิดคนละวิธีสิ้นเชิง** — จับให้ได้ตั้งแต่แรกแล้วที่เหลือง่าย · กลุ่ม "รายวัน" (`sharpe` · `annual_return` · `max drawdown`) คิดจากคอลัมน์ทั้งคอลัมน์รวดเดียว ไม่สนว่าเทรดไหนเป็นเทรดไหน · กลุ่ม "รายเทรด" (`win_rate` · `profit_factor` · `n trades` · `avg_holding_days`) ต้องรู้ก่อนว่า "เทรดหนึ่งครั้ง" เริ่มตรงไหนจบตรงไหน จึงต้องมีลูปจับคู่วันเข้ากับวันออก · ลูป `for` ตัวเดียวในฟังก์ชันนี้มีไว้เพื่อเรื่องนี้เท่านั้น

```
def compute_metrics(
    result_df: pd.DataFrame,
    start_equity: float = 100.0,
) -> dict:
    """
    คำนวณ performance metrics จาก backtest result

    Parameters
    -----
    result_df : pd.DataFrame  output จาก run_backtest
    start_equity : float      starting equity (default 100)

    Returns
    -----
    dict: sharpe, annual_return, max_drawdown, win_rate,
          profit_factor, n_trades, avg_holding_days
    """
    df = result_df.copy()

    # Sharpe เฉพาะวันที่ active (position != 0)
    active_pnl = df["daily_pnl"][df["position"] != 0].dropna()
```

```

if len(active_pnl) > 1:
    sharpe = (active_pnl.mean() / active_pnl.std()) * np.sqrt(252)
else:
    sharpe = float("nan")

# Annual return
valid_equity = df["equity"].dropna()
n_days = len(valid_equity)
if n_days > 1:
    total_return = valid_equity.iloc[-1] / start_equity - 1
    annual_return = (1 + total_return) ** (252 / n_days) - 1
else:
    annual_return = float("nan")

# Max drawdown
equity = valid_equity.values
peak = np.maximum.accumulate(equity)
drawdown = (equity - peak) / peak
max_dd = float(drawdown.min())

# Identify individual trades (position changes: 0-1 and 1-0)
pos_diff = df["position"].diff().fillna(0)
entry_dates = df.index[pos_diff == 1].tolist()
exit_dates = df.index[pos_diff == -1].tolist()

# match entries and exits
trade_pnls = []
holding_days = []
for i, entry in enumerate(entry_dates):
    exits_after = [e for e in exit_dates if e > entry]
    if not exits_after:
        continue
    exit_date = exits_after[0]
    trade_slice = df.loc[entry:exit_date, "daily_pnl"].dropna()
    trade_pnl = trade_slice.sum()
    trade_pnls.append(trade_pnl)
    holding_days.append(len(trade_slice))

n_trades = len(trade_pnls)
win_rate = (
    sum(1 for p in trade_pnls if p > 0) / n_trades
    if n_trades > 0 else float("nan")
)

wins = sum(p for p in trade_pnls if p > 0)
losses = abs(sum(p for p in trade_pnls if p <= 0))
profit_factor = wins / losses if losses > 0 else float("inf")

avg_holding = (
    sum(holding_days) / len(holding_days)
    if holding_days else float("nan")
)

return {
    "sharpe": round(sharpe, 3),
    "annual_return": round(annual_return, 4),
    "max_drawdown": round(max_dd, 4),
    "win_rate": round(win_rate, 3) if not pd.isna(win_rate) else float("nan"),
    "profit_factor": round(profit_factor, 3),
    "n_trades": n_trades,
    "avg_holding_days": round(avg_holding, 1) if not pd.isna(avg_holding) else float("nan"),
}

```



```
# ทดสอบ Round 2
metrics = compute_metrics(result)
print("=== Round 2 Metrics ===")
for k, v in metrics.items():
    if isinstance(v, float) and not pd.isna(v):
        if "return" in k or "drawdown" in k or "rate" in k:
            print(f" {k:20s}: {v:.1%}")
        else:
            print(f" {k:20s}: {v:.3f}")
    else:
        print(f" {k:20s}: {v}")
```

► OUTPUT

```
=== Round 2 Metrics ===
sharpe           : -1.681
annual_return    : -8.4%
max_drawdown     : -23.4%
win_rate         : 16.7%
profit_factor    : 0.150
n_trades         : 6
avg_holding_days : 19.800
```

🛠️ 3 ท่าที่ใช้ซ้ำได้ทุกครั้งที่ต้องวัดผลกลยุทธ์

```
active_pnl = df["daily_pnl"][df["position"] != 0].dropna() # ① กรองด้วย boolean

peak      = np.maximum.accumulate(equity)                # ② ยอดสูงสุดที่เคยไปถึง
drawdown  = (equity - peak) / peak

pos_diff   = df["position"].diff().fillna(0)              # ③ หาจุดเปลี่ยนสถานะ
entry_dates = df.index[pos_diff == 1].tolist()
exit_dates  = df.index[pos_diff == -1].tolist()
```

1. `df["daily_pnl"][df["position"] != 0]` — ข้างในวงเล็บเหลี่ยมไม่ใช่เลขลำดับ แต่เป็น **Series** ของ **True/False** ยาวเท่ากัน · pandas จะคืนเฉพาะแถวที่เป็น True · อ่านออกเสียงว่า "เอา daily_pnl เฉพาะวันที่ position ไม่ใช่ศูนย์" · **ทำไมต้องกรอง** — วันที่ไม่ได้ถืออะไร ถ้าเป็น 0 เป๊ะ ถ้านับรวมเข้าไป ส่วนเบี่ยงเบนมาตรฐานจะถูกเจือจางจนดูนิ่งผิดปกติ แล้ว Sharpe จะสูงเกินจริง
2. `np.maximum.accumulate` — "ยอดสูงสุดที่เคยไปถึง ณ แต่ละจุด" เช่น `[100, 105, 103, 108]` ได้ `[100, 105, 105, 108]` · ต่างจาก `.max()` ที่ให้เลขเดียว · พอเอา equity ปัจจุบันลบ peak แล้วหารด้วย peak ก็ได้ **drawdown ณ ทุกวันที่** · `.min()` ของมันคือจุดที่เจ็บที่สุดที่เคยผ่านมา
3. `position.diff()` คู่กับ `df.index[...]` — `diff()` ได้ `+1` ตอนเข้า `-1` ตอนออก · `df.index[เงื่อนไข]` คืนวันที่ของแถวที่ตรงเงื่อนไข ไม่ใช่ค่าในแถว · เป็นท่ามาตรฐานสำหรับคำถาม "มันเกิดขึ้นเมื่อไหร่บ้าง"

ส่วนลูปจับคู่ข้างล่าง (`exits_after = [e for e in exit_dates if e > entry]`) มี**สมมติฐาน**ซ่อนอยู่ที่เราควรเห็น: มันเลือกวันออกตัวแรกที่มาหลังวันเข้า ซึ่งถูกต้องเฉพาะกับกลยุทธ์ที่ถือได้ทีละหนึ่งไม้เท่านั้น · ถ้าเปลี่ยนไปทำกลยุทธ์ที่เข้าซ้อนหลายไม้ ฟังก์ชันนี้จะนับผิดโดยไม่ error

30.5 Round 3 — Walk-Forward Validation

📖 **อ่านว่า:** walk-forward คือการเลียนแบบสิ่งที่เกิดขึ้นจริงตามเวลา · ใช้อัตตคติสนใจ แล้วดูว่าอนาคตที่ยังไม่เคยเห็นให้ผลอย่างไร · แล้วเลื่อนหน้าต่างไปข้างหน้าทำซ้ำ · โค้ดข้างล่างยาวก็จริงแต่ใครมีแค่ while วงเดียวที่เลื่อน start ทีละ `step_size` · ในแต่ละรอบมีของ 3 อย่าง: ตัดช่วงข้อมูล → รัน backtest → เก็บตัวเลขลง `results` · ตอนจบแปลง `results` เป็น DataFrame ที่เดียว — เป็นแพทเทิร์นที่จะเจอซ้ำไปตลอด: สะสมลง list ก่อน แล้วค่อยสร้าง DataFrame ครั้งเดียวตอนท้าย เพราะการต่อ DataFrame ทีละแถวช้ากว่ามาก

ROUND 3 PROMPT

Task: เพิ่ม walk-forward validation บน volatility mean-reversion strategy ปัญหา Round 2: Sharpe -1.681 วัดจาก in-sample ชุดเดียว – ต้องตรวจ out-of-sample ว่าผลนี้คงที่ข้ามช่วงเวลาใหม่ หรือแค่บังเอิญของ sample นี้ Input: - prices: pd.Series - train_size: int = 252 (1 ปีแรก train) - test_size: int = 63 (3 เดือน test) - step_size: int = 63 (เลื่อน 3 เดือนทีละรอบ) - params: dict = {'vol_window': 21, 'zscore_window': 63, 'entry_z': 1.5, 'exit_z': 0.5} Output: - pd.DataFrame: fold | train_start | train_end | test_start | test_end | sharpe_train | sharpe_test | n_trades Constraints: - parameter ต้อง fit บน train window เท่านั้น - test window ต้องไม่ overlap กับ train window - vol_zscore ในแต่ละ fold ต้องคำนวณ fresh บน train data นั้น - report mean และ std ของ test Sharpe ข้าม folds Libraries: pandas, numpy

```
def walk_forward_validation(
    prices: pd.Series,
    train_size: int = 252,
    test_size: int = 63,
    step_size: int = 63,
    params: dict = None,
) -> pd.DataFrame:
    """
    Walk-forward validation สำหรับ vol mean-reversion strategy

    Parameters
    -----
    prices : pd.Series    daily close prices
    train_size : int      training window (days)
    test_size : int       test window (days)
    step_size : int       rolling step (days)
    params : dict         strategy parameters

    Returns
    -----
    pd.DataFrame: fold results with in/out-of-sample Sharpe
    """
    if params is None:
        params = {
            "vol_window": 21,
            "zscore_window": 63,
            "entry_z": 1.5,
            "exit_z": 0.5,
        }

    results = []
    n = len(prices)
    fold = 0

    start = 0
    while start + train_size + test_size <= n:
        train_end = start + train_size
        test_end = train_end + test_size

        train_prices = prices.iloc[start:train_end]
        test_prices = prices.iloc[train_end:test_end]

        # ---- คำนวณ feature ครั้งเดียวบน "ประวัติต่อเนื่อง" train+test ----
        # ⚠️ ห้ามคำนวณ vol_zscore บน test window ล้วน ๆ . เหตุผล 2 ข้อ:
        # 1) ผิดหลัก – วันแรกของ test ในชีวิตจริงเรามีข้อมูล train
        #    อยู่ในมือแล้ว การทิ้งไม่ใช้การกั้นมองอนาคต แต่คือการโยน
        #    ข้อมูลที่มีสิทธิ์ใช้ทิ้งเปล่า ๆ
        # 2) พังเจียบ – test_size (63) ไม่พอเลี้ยง zscore_window (63)
        #    เพราะ ann_vol กิน 20 แถวแรกไปเป็น NaN แล้ว
        #    → vol_zscore เป็น NaN ทั้งชุด → ไม่มีสัญญาณ → 0 เทวด

        # ... (calculations and Sharpe calculation) ...

        results.append(...)
        fold += 1
        start = start + step_size

    return pd.DataFrame(results)
```

```

# โค้ดไม่ error เลย แต่ผลลัพธ์เป็น NaN ทุก fold
# ยังไม่มองขนาดคดียุติ เพราะ rolling ทุกตัวเป็น center=False
hist_prices = prices.iloc[start:test_end]

vdf_hist = compute_vol_zscore(
    hist_prices,
    vol_window=params["vol_window"],
    zscore_window=params["zscore_window"],
)
sig_hist = generate_signal(
    vdf_hist["vol_zscore"],
    entry_z=params["entry_z"],
    exit_z=params["exit_z"],
)
res_hist = run_backtest(hist_prices, sig_hist)

# ---- ตัดผลออกเป็นช่วง train / test แล้วค่อยวัดแยกกัน ----
res_train = res_hist.iloc[:train_size]
res_test = res_hist.iloc[train_size:]

m_train = compute_metrics(res_train)
# equity ของช่วง test ไม่ได้เริ่มที่ 100 (มันต่อมาจาก train)
# ต้องบอก start_equity ให้ตรง ไม่งั้น annual_return จะเพี้ยน
m_test = compute_metrics(
    res_test, start_equity=res_test["equity"].iloc[0]
)

```

```

results.append({
    "fold": fold,
    "train_start": train_prices.index[0].date(),
    "train_end": train_prices.index[-1].date(),
    "test_start": test_prices.index[0].date(),
    "test_end": test_prices.index[-1].date(),
    "sharpe_train": m_train["sharpe"],
    "sharpe_test": m_test["sharpe"],
    "n_trades_test": m_test["n_trades"],
})

```

```

fold += 1
start += step_size

```

```

return pd.DataFrame(results)

```

```

# สร้างข้อมูล 3 ปี สำหรับ walk-forward
np.random.seed(42)
n_wf = 252 * 3
prices_wf = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0003, 0.015, n_wf))),
    index=pd.date_range("2021-01-01", periods=n_wf)
)

wf_results = walk_forward_validation(prices_wf)
print("=== Walk-Forward Results ===")
print(wf_results[["fold", "train_start", "test_start",
    "sharpe_train", "sharpe_test",
    "n_trades_test"]].to_string(index=False))
print(f"\nMean test Sharpe: {wf_results['sharpe_test'].mean():.3f}")
print(f"Std test Sharpe: {wf_results['sharpe_test'].std():.3f}")
print(f"% folds > 0: "
    f"{(wf_results['sharpe_test'] > 0).mean():.0%}")

```

► OUTPUT

=== Walk-Forward Results ===

fold	train_start	test_start	sharpe_train	sharpe_test	n_trades_test
0	2021-01-01	2021-09-10	-1.330	0.843	1
1	2021-03-05	2021-11-12	0.496	NaN	0
2	2021-05-07	2022-01-14	0.496	-0.400	1
3	2021-07-09	2022-03-18	-0.589	-3.872	1
4	2021-09-10	2022-05-20	-1.488	NaN	0
5	2021-11-12	2022-07-22	-2.759	1.084	1
6	2022-01-14	2022-09-23	0.367	0.626	1
7	2022-03-18	2022-11-25	0.877	-2.791	0

Mean test Sharpe: -0.752

Std test Sharpe: 2.089

% folds > 0: 38%

การตีความผล Walk-Forward — คำตอบคือ "อย่าเอาไปใช้"

- **Mean test Sharpe -0.752** — ติดลบ · ไม่มีอะไรให้ deploy
- **Std ข้าม fold = 2.089** — ค่าเหวี่ยงตั้งแต่ +1.084 ถึง -3.872 · ส่วนเบี่ยงเบนใหญ่กว่าค่าเฉลี่ยเกือบ 3 เท่า แปลว่าตัวเลขเฉลี่ยไม่มีความหมาย ต่อให้มันบังเอิญเป็นบวกก็ตาม
- **n_trades_test = 0 หรือ 1 ในทุก fold · รวมทั้งหมด 5 เทวดใน 2 ปี** — นี่คือข้อที่สำคัญที่สุด · Sharpe ที่คำนวณจากเทรดเดียวไม่ใช่ Sharpe มันคือผลของเทรดนั้นเทรดเดียว · fold 3 ได้ -3.872 เพราะเทรดเดียวนั้นขาดทุนไม่ได้แปลว่ากลยุทธ์แยกว่า fold อื่น 4 เท่า
- **2 fold ได้ NaN** — ไม่มีวันไหนถือ position เลย · Sharpe จึงคำนวณไม่ได้ · NaN ตรงนี้ไม่ใช่ bug แต่เป็นข้อมูลจริงว่า "ไตรมาสนั้นไม่มีสัญญาณเลย"

ข้อสรุปที่ถูกต้องคือหยุด — ไม่ใช่ไปปรับ entry_z จนกว่าตัวเลขจะสวย · การกลับไปหมุนพารามิเตอร์หลังเห็นผล test แล้วคือ **overfitting** ที่ใช้ test set เป็นเชื้อเพลิง ซึ่งเป็นบาปข้อที่บทที่ 28 เตือนไว้ทั้งบท

ทำไมบทนี้จบด้วยกลยุทธ์ที่ใช้ไม่ได้ — และทำไมนั่นคือของจริง

ราคาที่ใช้ทดสอบทั้งบทสร้างจาก np.random.normal — เป็น **random walk** บริสุทธิ์ ที่ไม่มีรูปแบบให้จับตั้งแต่แรก · ดังนั้นผลที่ถูกต้องเพียงผลเดียวคือ "ไม่มี edge" · ถ้าตัวเลขออกมาสวยนั้นต่างหากที่ควรทำให้ตกใจ เพราะแปลว่าเครื่องมือวัดของเราเสีย

สังเกตลำดับที่เกิดขึ้นในบทนี้ ซึ่งเหมือนกับที่เกิดในงานจริงทุกประการ:

1. **Round 1** — โค้ดรันได้ equity curve ออกมาเป็นเส้นสวย ๆ · ดูเหมือนมีอะไร
2. **Round 2** — พรวัดด้วย metric ที่ถูกต้อง Sharpe ออกมา -1.681 · เริ่มเห็นความจริง
3. **Round 3** — walk-forward ยืนยันว่าไม่มีอะไรเลย และบอกเพิ่มด้วยว่าข้อมูลน้อยเกินกว่าจะสรุปอะไรได้ตั้งแต่ต้น (5 เทวดใน 2 ปี)

คุณค่าของ walk-forward ไม่ใช่การรับรองกลยุทธ์ แต่คือการห้ามไม่ให้ deploy ของที่ยังไม่ได้พิสูจน์ · ในอาชีพนี้ไฉ่เดียวส่วนใหญ่ตายตรงนี้ และนั่นคือการทำงานที่ถูกต้องของเครื่องมือ

● สิ่งที่ต้องแก้ก่อน ถ้าจะทำ walk-forward นี้ให้ตัดสินใจอะไรได้จริง

ปัญหาที่แท้จริงไม่ใช่ "Sharpe ติดลบ" แต่คือ การออกแบบ fold ไม่มีกำลังทางสถิติที่จะตอบคำถามอะไรได้เลย · กลยุทธ์เทรดราว 2-3 ครั้งต่อปี แต่หน้าต่าง test ยาวแค่ 63 วัน — โดยเฉลี่ยจึงได้ไม่ถึง 1 เทรดต่อ fold · ไม่ว่าผลจะออกมาบวกหรือลบก็ตีความไม่ได้ทั้งคู่

ทางแก้เรียงตามลำดับที่ควรลอง:

1. **ขยาย test_size ให้กินอย่างน้อย ~30 เทรด** — สำหรับกลยุทธ์ความถี่นี้แปลว่าหน้าต่างระดับหลายปี ซึ่งแปลว่าข้อมูล 3 ปีไม่พอทำ walk-forward ตั้งแต่ต้น · รู้ข้อนี้ก่อนเขียนโค้ดประหยัดเวลาไปทั้งสัปดาห์
2. **เปลี่ยนไปวัดสิ่งที่มีตัวอย่างเยอะกว่า** — เช่นวัดที่ระดับ "วันที่ถือ position" แทน "เทรด" หรือใช้ bootstrap บนผลตอบแทนรายวันเพื่อหาช่วงความเชื่อมั่นของ Sharpe แทนที่จะดูค่าเดียว
3. **ลดความถี่ของคำถาม** — ถ้าข้อมูลตอบได้แค่ "กลยุทธ์นี้แย่ชัด ๆ ไหม" ก็อย่าไปถามมันว่า "Sharpe เท่าไร"

กฎที่ใช้ได้ตลอดชีวิต: ก่อนรัน walk-forward ให้ประมาณจำนวนเทรดต่อ fold ก่อน · ถ้าได้ตัวเลขหลักหน่วย ผลลัพธ์จะเป็นเสียงรบกวนล้วน ๆ ไม่
ว่าโค้ดจะถูกแก้ไขไหม

30.6 สรุป: Final Strategy Code (โครงที่พร้อม — ยังไม่ใช้กลยุทธ์ที่พร้อม)

โค้ดข้างล่างคือการรวมทุกอย่างในบทนี้เป็นคลาสเดียว · สิ่งที่ "พร้อม" คือโครงสร้าง — แยก feature / signal / backtest ออกจากกัน มี parameter เป็น
dataclass เปลี่ยนค่าได้โดยไม่ต้องแก้โค้ด · ส่วนตัวกลยุทธ์เองยังสอบไม่ผ่าน ตามผล walk-forward ข้างบน · อ่านโค้ดนี้ในฐานะแม่แบบที่จะเอาไอเดียตัว
ถัดไปมาใส่ไม่ใช่ของที่เอาไปเปิดเทรดได้

```
# Final integrated strategy — หลัง 3 รอบ iterate
import numpy as np
import pandas as pd
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class VolMeanReversionParams:
    """Parameters สำหรับ Volatility Mean-Reversion Strategy"""
    vol_window: int = 21
    zscore_window: int = 63
    entry_z: float = 1.5
    exit_z: float = 0.5
    fee_per_trade: float = 0.001
    max_position: float = 1.0 # fraction ของ capital

class VolMeanReversionStrategy:
    """
    Volatility Mean-Reversion Strategy

    Logic:
    - Long underlying เมื่อ realized vol spike สูงผิดปกติ
    - Exit เมื่อ vol กลับสู่ระดับปกติ

    สถานะการตรวจ (อย่าลบบรรทัดพวกนี้ — มันคือเหตุผลที่ยังไม่ deploy):
    - Embargo test: PASS ไม่มี lookahead bias
    - Walk-forward: FAIL mean test Sharpe -0.752, ผ่านแค่ 38% ของ fold
      และมีแค่ 5 เทรดใน 2 ปี = สรุปอะไรไม่ได้
    - ข้อสรุป: โครงสร้างใช้ได้ · กลยุทธ์ยังไม่ผ่าน · ห้ามต่อเข้าเงินจริง
    - Transaction costs included
    """

    def __init__(self, params: Optional[VolMeanReversionParams] = None):
        self.params = params or VolMeanReversionParams()

    def compute_features(self, prices: pd.Series) -> pd.DataFrame:
        """คำนวณ features — no lookahead bias"""
        p = self.params
        log_ret = np.log(prices / prices.shift(1))
        ann_vol = (
            log_ret.rolling(p.vol_window, min_periods=p.vol_window).std()
            * np.sqrt(252) * 100
        )
        roll_mean = ann_vol.rolling(p.zscore_window, min_periods=p.zscore_window).mean()
        roll_std = ann_vol.rolling(p.zscore_window, min_periods=p.zscore_window).std()
        vol_zscore = (ann_vol - roll_mean) / roll_std

        return pd.DataFrame({
```

```

        "log_ret":    log_ret,
        "ann_vol":    ann_vol,
        "vol_zscore": vol_zscore,
    })

def signal(self, features: pd.DataFrame) -> pd.Series:
    """สร้าง signal – ใช้เฉพาะข้อมูลปัจจุบัน"""
    p    = self.params
    z    = features["vol_zscore"]
    sig = pd.Series("HOLD", index=z.index)
    sig[z > p.entry_z]    = "LONG"
    sig[z.abs() < p.exit_z] = "EXIT"
    sig[z.isna()]         = "HOLD"
    return sig

def backtest(self, prices: pd.Series) -> dict:
    """รัน full backtest และคืน metrics + equity curve"""
    features = self.compute_features(prices)
    signals  = self.signal(features)
    result   = run_backtest(prices, signals, self.params.fee_per_trade)
    metrics  = compute_metrics(result)
    return {"metrics": metrics, "equity": result["equity"],
            "signals": signals, "features": features}

def live_signal(self, prices: pd.Series) -> str:
    """สำหรับ live trading – คืน signal ล่าสุด"""
    features = self.compute_features(prices)
    signals  = self.signal(features)
    return signals.iloc[-1]

# ใช้งาน
strategy = VolMeanReversionStrategy()
result   = strategy.backtest(prices_wf)

print("=== Final Strategy Metrics (In-Sample) ===")
for k, v in result["metrics"].items():
    if isinstance(v, float) and not pd.isna(v):
        if "return" in k or "drawdown" in k or "rate" in k:
            print(f" {k:22s}: {v:.1%}")
        else:
            print(f" {k:22s}: {v:.3f}")
    else:
        print(f" {k:22s}: {v}")

# Live signal ปัจจุบัน
print(f"\nCurrent signal: {strategy.live_signal(prices_wf)}")

```

► OUTPUT

```

=== Final Strategy Metrics (In-Sample) ===
sharpe           : -0.947
annual_return    : -3.6%
max_drawdown     : -15.4%
win_rate         : 40.0%
profit_factor    : 0.275
n_trades         : 10
avg_holding_days : 17.100
Current signal: LONG

```

Round	สิ่งที่เพิ่ม/แก้	ผลลัพธ์
Round 1	Core algorithm + embargo test	ไม่มี lookahead — แต่หน่วงเกินไป 1 แท่ง (เจอตอน audit)
Round 2	Active-days Sharpe, trade-level win rate, profit factor	วัดถูกขึ้น แล้วเห็นว่า Sharpe ตีกลับ
Round 3	Walk-forward validation (8 folds)	ครบ — mean test Sharpe $-0.752 \cdot 5$ เทวดาใน 2 ปี

อ่านตารางนี้ตามลำดับจะเห็นสิ่งที่บ่นบ่อยๆ: **ทุกรอบทำให้เห็นความจริงชัดขึ้น และความจริงก็แย่งเรื่อย ๆ** . นั่นคือรูปร่างปกติของงานวิจัยกลยุทธ์ที่ทำถูกวิธี . รอบที่ทำให้ตัวเลขดีขึ้นเรื่อย ๆ ต่างหากที่ควรสงสัยตัวเอง

► **แบบฝึกหัด:** เพิ่ม regime filter — เทวดาเฉพาะเมื่อ 200-day trend เป็น uptrend

จบ Part V — สิ่งที่ได้จากการเรียนรู้ AI-Assisted Coding

- บท 26:** Prompt template 4 ส่วน ช่วยให้ AI สร้างโค้ดที่ตรงความต้องการมากขึ้น ✓
- บท 27:** Review checklist 8 ข้อ + embargo test ตรวจ lookahead ทุกครั้ง ✓
- บท 28:** Red flags 5 ข้อ — shift(1), scaler leakage, transaction costs, p-hacking ✓
- บท 29:** Prompt templates เฉพาะสำหรับ OLS, ADF, z-score, scipy.optimize ✓
- บท 30:** 3 รอบ iterate: generate → audit → improve → walk-forward ✓ (และจบด้วยการปฏิเสธกลยุทธ์ — ผลลัพธ์ที่พบบ่อยที่สุดในงานจริง)

AI เป็นผู้ช่วยที่ทรงพลัง แต่ **ความรับผิดชอบต่อความถูกต้องทางคณิตศาสตร์** ยังอยู่ที่เรา — ตรวจทุกบรรทัดที่ AI สร้าง ใช้ embargo test และ walk-forward validation เสมอ

ใน **Part VI** เราจะนำทุกอย่างที่เรียนมาสร้างระบบ production-grade: live data ingestion, order management, risk monitoring แบบ real-time